



Analisi e Progetto di Algoritmi

Laurea Triennale in Informatica
(codice: E3101Q113)

AA 2025/2026

Algoritmo di Dijkstra
(cammini minimi da sorgente)

Docente: Raffaella Rizzi

[Cammini minimi da sorgente]

INPUT

- ✓ Grafo $G = (V, E)$ orientato
- ✓ $W : E \rightarrow \mathbb{R}^+$ tale che $W(u,v)$ è il peso dell'arco (u,v)
- ✓ sorgente $s \in V$

OUTPUT

Per ogni $v \in V$, trovare un cammino P_{sv} di peso minimo $\delta(s,v)$ dal vertice s al vertice v

SOLUZIONE: algoritmo di Dijkstra



[Sottostruttura ottima]

Sia $P_{sv} = \langle s, \dots, u, v \rangle$ un cammino minimo da s a v



Il sottocammino $\langle s, \dots, u \rangle$ è il cammino minimo da s a u

$$\Rightarrow \delta(s, v) = \delta(s, u) + W(u, v)$$

[Tecnica del rilassamento]

Due attributi per ogni vertice $x \in V$:

- $\mathbf{x.d} \geq \delta(s,x)$, limite superiore per $\delta(s,x)$

Inizializzazione

- ✓ $\forall x \in V \setminus \{s\}, \mathbf{x.d} = +\infty$
- ✓ $\mathbf{x.d} = 0$ se $x = s$



s

x

Vengono eseguiti $|E|$ passi di rilassamento

[Tecnica del rilassamento]

Due attributi per ogni vertice $x \in V$:

- $\mathbf{x.d} \geq \delta(s,x)$, limite superiore per $\delta(s,x)$

Inizializzazione

- ✓ $\forall x \in V \setminus \{s\}, \mathbf{x.d} = +\infty$
- ✓ $\mathbf{x.d} = 0$ se $x = s$



s

x

Vengono eseguiti $|E|$ passi di rilassamento

→ primo passo di rilassamento

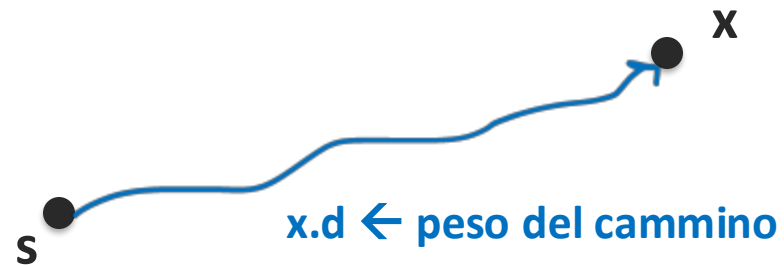
[Tecnica del rilassamento]

Due attributi per ogni vertice $x \in V$:

- $x.d \geq \delta(s,x)$, limite superiore per $\delta(s,x)$

Inizializzazione

- ✓ $\forall x \in V \setminus \{s\}, x.d = +\infty$
- ✓ $x.d = 0$ se $x = s$



Vengono eseguiti $|E|$ passi di rilassamento

→ secondo passo di rilassamento

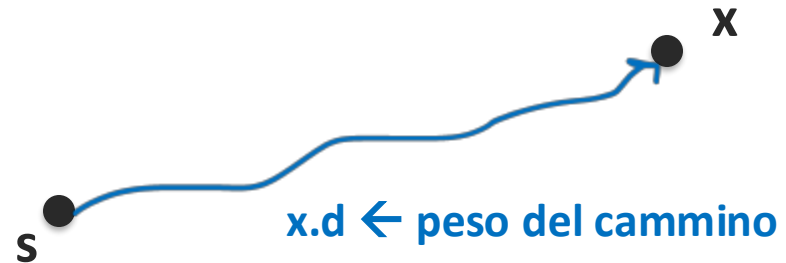
[Tecnica del rilassamento]

Due attributi per ogni vertice $x \in V$:

- $x.d \geq \delta(s,x)$, limite superiore per $\delta(s,x)$

Inizializzazione

- ✓ $\forall x \in V \setminus \{s\}, x.d = +\infty$
- ✓ $x.d = 0$ se $x = s$



Vengono eseguiti $|E|$ passi di rilassamento

→ terzo passo di rilassamento

[Tecnica del rilassamento]

Due attributi per ogni vertice $x \in V$:

- $x.d \geq \delta(s,x)$, limite superiore per $\delta(s,x)$

Inizializzazione

- ✓ $\forall x \in V \setminus \{s\}, x.d = +\infty$
- ✓ $x.d = 0$ se $x = s$



Vengono eseguiti $|E|$ passi di rilassamento

→ quarto passo di rilassamento

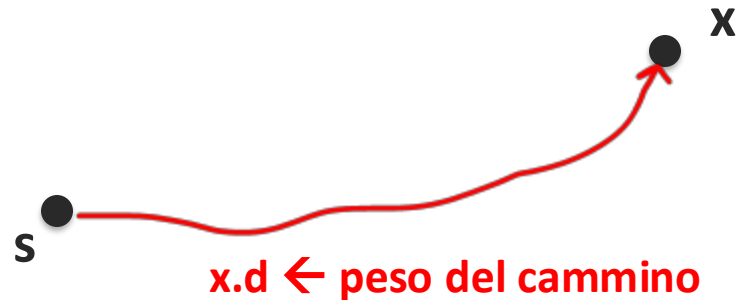
[Tecnica del rilassamento]

Due attributi per ogni vertice $x \in V$:

- $x.d \geq \delta(s,x)$, limite superiore per $\delta(s,x)$

Inizializzazione

- ✓ $\forall x \in V \setminus \{s\}, x.d = +\infty$
- ✓ $x.d = 0$ se $x = s$



Vengono eseguiti $|E|$ passi di rilassamento

→ quinto passo di rilassamento

[Tecnica del rilassamento]

Due attributi per ogni vertice $x \in V$:

- $x.d \geq \delta(s,x)$, limite superiore per $\delta(s,x)$

Inizializzazione

- ✓ $\forall x \in V \setminus \{s\}, x.d = +\infty$
- ✓ $x.d = 0$ se $x = s$



Vengono eseguiti $|E|$ passi di rilassamento

→ sesto passo di rilassamento

[Tecnica del rilassamento]

Due attributi per ogni vertice $x \in V$:

- $x.d \geq \delta(s,x)$, limite superiore per $\delta(s,x)$

Inizializzazione

- ✓ $\forall x \in V \setminus \{s\}, x.d = +\infty$
- ✓ $x.d = 0$ se $x = s$



Vengono eseguiti $|E|$ passi di rilassamento

→ settimo passo di rilassamento

[Tecnica del rilassamento]

Due attributi per ogni vertice $x \in V$:

- $x.d \geq \delta(s,x)$, limite superiore per $\delta(s,x)$

Inizializzazione

- ✓ $\forall x \in V \setminus \{s\}, x.d = +\infty$
- ✓ $x.d = 0$ se $x = s$



Vengono eseguiti $|E|$ passi di rilassamento

→ etc. etc.

[Tecnica del rilassamento]

Due attributi per ogni vertice $x \in V$:

- $\mathbf{x.d} \geq \delta(s,x)$, limite superiore per $\delta(s,x)$

Inizializzazione

- ✓ $\forall x \in V \setminus \{s\}, \mathbf{x.d} = +\infty$
- ✓ $\mathbf{x.d} = 0$ se $x = s$



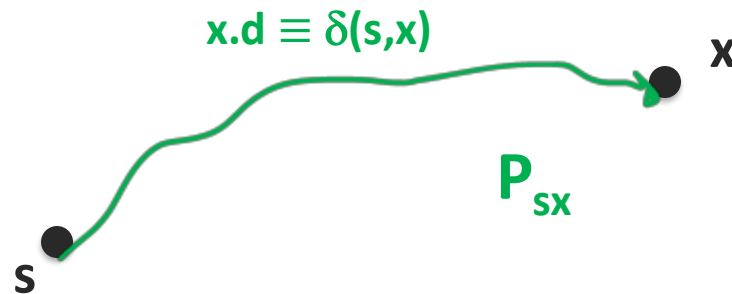
[Tecnica del rilassamento]

Due attributi per ogni vertice $x \in V$:

- $x.d \geq \delta(s,x)$, limite superiore per $\delta(s,x)$

Inizializzazione

- ✓ $\forall x \in V \setminus \{s\}, x.d = +\infty$
- ✓ $x.d = 0$ se $x = s$



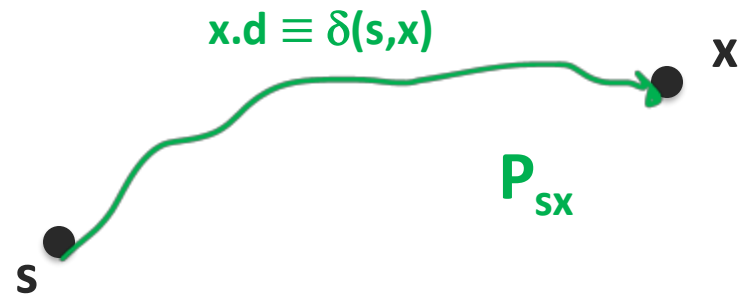
[Tecnica del rilassamento]

Due attributi per ogni vertice $x \in V$:

- $\mathbf{x.d} \geq \delta(s,x)$, limite superiore per $\delta(s,x)$

Inizializzazione

- ✓ $\forall x \in V \setminus \{s\}, \mathbf{x.d} = +\infty$
- ✓ $\mathbf{x.d} = 0$ se $x = s$



Al termine

- ✓ $\forall x \in V, \mathbf{x.d} = \delta(s,x)$

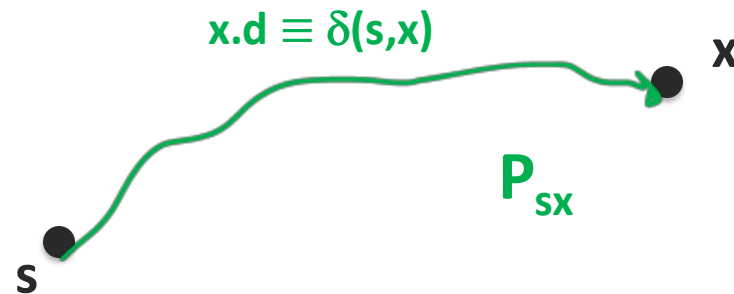
[Tecnica del rilassamento]

Due attributi per ogni vertice $x \in V$:

- $\mathbf{x.d} \geq \delta(s,x)$, limite superiore per $\delta(s,x)$
- $\mathbf{x.\pi}$, predecessore di x nel cammino di peso $\mathbf{x.d}$

Inizializzazione

- ✓ $\forall x \in V \setminus \{s\}, \mathbf{x.d} = +\infty$
- ✓ $\mathbf{x.d} = 0$ se $x = s$



Al termine

- ✓ $\forall x \in V, \mathbf{x.d} = \delta(s,x)$



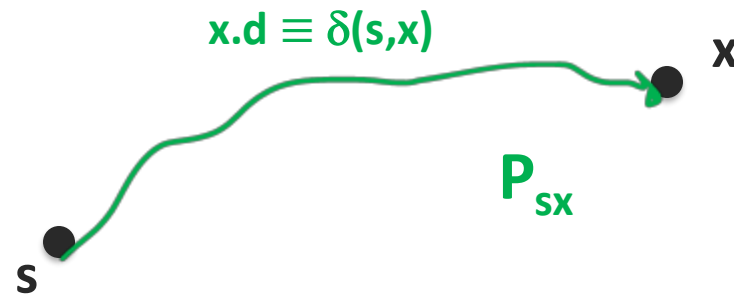
[Tecnica del rilassamento]

Due attributi per ogni vertice $x \in V$:

- $\mathbf{x.d} \geq \delta(s,x)$, limite superiore per $\delta(s,x)$
- $\mathbf{x.\pi}$, predecessore di x nel cammino di peso $\mathbf{x.d}$

Inizializzazione

- ✓ $\forall x \in V \setminus \{s\}, \mathbf{x.d} = +\infty$
- ✓ $\mathbf{x.d} = 0$ se $x = s$
- ✓ $\forall x \in V, \mathbf{x.\pi} = \text{NIL}$



Al termine

- ✓ $\forall x \in V, \mathbf{x.d} = \delta(s,x)$



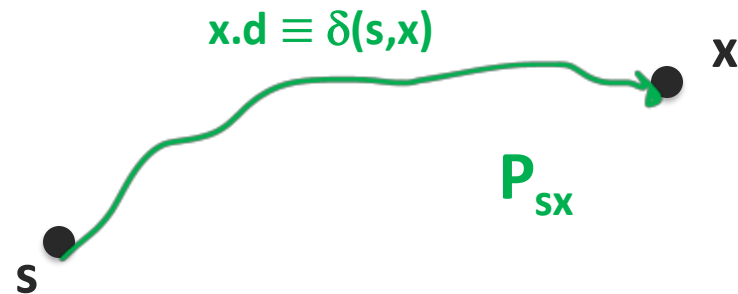
[Tecnica del rilassamento]

Due attributi per ogni vertice $x \in V$:

- $\mathbf{x.d} \geq \delta(s, x)$, limite superiore per $\delta(s, x)$
- $\mathbf{x.\pi}$, predecessore di x nel cammino di peso $\mathbf{x.d}$

Inizializzazione

- ✓ $\forall x \in V \setminus \{s\}, \mathbf{x.d} = +\infty$
- ✓ $\mathbf{x.d} = 0$ se $x = s$
- ✓ $\forall x \in V, \mathbf{x.\pi} = \text{NIL}$



Al termine

- ✓ $\forall x \in V, \mathbf{x.d} = \delta(s, x)$
- ✓ $\forall x \in V, \mathbf{x.\pi}$ è il predecessore di x in $\mathbf{P_{sx}}$

[Tecnica del rilassamento]

Due attributi per ogni vertice $x \in V$:

- $x.d \geq \delta(s,x)$, limite superiore per $\delta(s,x)$
- $x.\pi$, predecessore di x nel cammino di peso $x.d$

INITIALIZE-SINGLE-SOURCE ($G=(V,E)$, s)

foreach $x \in V$ **do**

$x.d \leftarrow +\infty$

$x.\pi \leftarrow \text{NIL}$

$s.d \leftarrow 0$

Tecnica del rilassamento

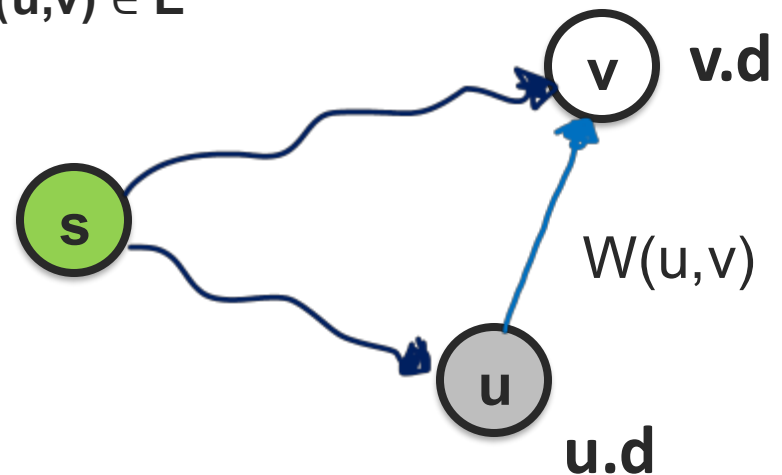
Due attributi per ogni vertice $x \in V$:

- $x.d \geq \delta(s,x)$, limite superiore per $\delta(s,x)$
- $x.\pi$, predecessore di x nel cammino di peso $x.d$

PASSO DI RILASSAMENTO DI $(u,v) \in E$

RELAX (u, v, W)

```
if  $v.d > u.d + W(u,v)$  then  
     $v.d \leftarrow u.d + W(u,v)$   
     $v.\pi \leftarrow u$ 
```



Tecnica del rilassamento

Due attributi per ogni vertice $x \in V$:

- $x.d \geq \delta(s,x)$, limite superiore per $\delta(s,x)$
- $x.\pi$, predecessore di x nel cammino di peso $x.d$

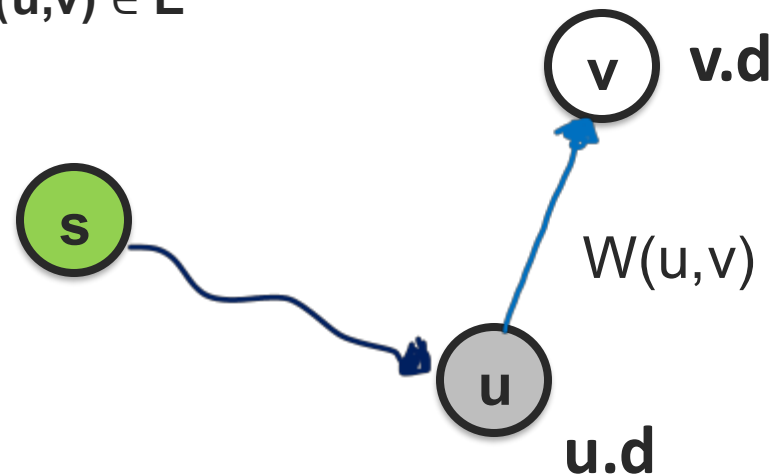
PASSO DI RILASSAMENTO DI $(u,v) \in E$

RELAX (u, v, W)

if $v.d > u.d + W(u,v)$ **then**

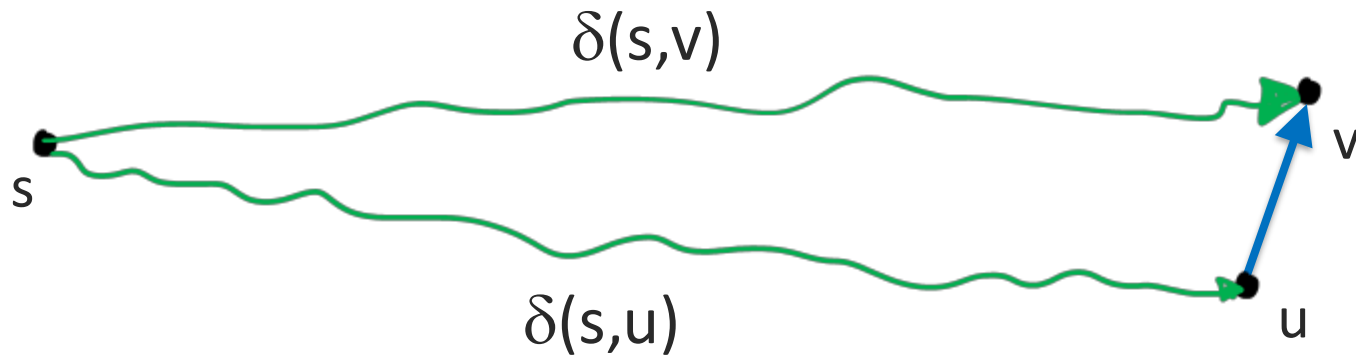
$\rightarrow v.d \leftarrow u.d + W(u,v) \leftarrow$

$v.\pi \leftarrow u$



[Disuguaglianza triangolare

$$\forall (u,v) \in E, \delta(s,v) \leq \delta(s,u) + W(u,v)$$



[Proprietà del limite superiore]

$\forall x \in V$, allora si ha che $x.d \geq \delta(s,x)$ e non cambia più non appena raggiunge il valore $\delta(s,x)$

[Proprietà del limite superiore]

$\forall x \in V$, allora si ha che $x.d \geq \delta(s,x)$ e non cambia più
non appena raggiunge il valore $\delta(s,x)$

[Proprietà del limite superiore]

$\forall x \in V$, allora si ha che $x.d \geq \delta(s,x)$ e non cambia più non appena raggiunge il valore $\delta(s,x)$

CASO BASE (inizializzazione)

$$s.d = 0 \Rightarrow s.d \geq \delta(s,s) = 0$$

$$\forall x \in V \setminus \{s\}, x.d = +\infty$$

$\Rightarrow x.d > \delta(s,x)$, se $\exists$ un cammino da s a x

$\Rightarrow x.d = \delta(s,x)$, se $\nexists$ un cammino da s a x

[Proprietà del limite superiore]

$\forall x \in V$, allora si ha che $x.d \geq \delta(s,x)$ e non cambia più non appena raggiunge il valore $\delta(s,x)$

PASSO INDUTTIVO

IPOTESI: $\forall x \in V$, $x.d \geq \delta(s,x)$ prima di **RELAX**(u, v, W)

TESI: $\forall x \in V$, $x.d \geq \delta(s,x)$ dopo **RELAX**(u, v, W)

E' sufficiente dimostrare che $v.d \geq \delta(s,v)$

$$v.d = u.d + W(u,v) \geq \delta(s,u) + W(u,v) \geq \delta(s,v)$$

$\Rightarrow v.d \geq \delta(s,v)$

```
if v.d > u.d + W(u,v) then
     $\Rightarrow v.d \leftarrow u.d + W(u,v)$ 
     $v.\pi \leftarrow u$ 
```

[Proprietà del limite superiore]

$\forall x \in V$, allora si ha che $x.d \geq \delta(s,x)$ e non cambia più
non appena raggiunge il valore $\delta(s,x)$

[Proprietà del limite superiore]

$\forall x \in V$, allora si ha che $x.d \geq \delta(s,x)$ e non cambia più non appena raggiunge il valore $\delta(s,x)$

Occorre dimostrare che una chiamata a **RELAX**(u, v, W) non è in grado, per un $x \in V$, di far variare $x.d$ se $x.d = \delta(s,x)$.

RELAX(u, v, W) può far variare $x.d$ solo se $v = x$.

$$x.d = \delta(s,x) \leq \delta(s,u) + W(u,x) \leq u.d + W(u,x)$$

$\Rightarrow x.d$ non varia

```
if x.d > u.d + W(u,x) then
  x.d ← u.d + W(u,x)
  x.π ← u
```

[Proprietà assenza cammino]

$\forall x \in V$, tale che $\nexists$ cammino da s a x , allora $x.d = \delta(s,x)$

Dopo l'inizializzazione si ha che:

$$x.d = +\infty = \delta(s,x)$$

Dopo ogni rilassamento, per la proprietà del limite superiore:

$$x.d = \delta(s,x)$$

[Disuguaglianza triangolare 2]

$\forall (u,v) \in E$, allora $v.d \leq u.d + W(u,v)$ dopo la chiamata a **RELAX**(u, v, W)

```
if  $v.d > u.d + W(u,v)$  then  
     $v.d \leftarrow u.d + W(u,v)$   
     $v.\pi \leftarrow u$ 
```

[Disuguaglianza triangolare 2]

$\forall (u,v) \in E$, allora $v.d \leq u.d + W(u,v)$ dopo la chiamata a **RELAX**(u, v, W)

Se prima della chiamata si ha:

$$v.d > u.d + W(u,v)$$

```
if  $v.d > u.d + W(u,v)$  then  
     $v.d \leftarrow u.d + W(u,v)$   
     $v.\pi \leftarrow u$ 
```

[Disuguaglianza triangolare 2]

$\forall (u,v) \in E$, allora $v.d \leq u.d + W(u,v)$ dopo la chiamata a **RELAX**(u, v, W)

Se prima della chiamata si ha:

$$v.d > u.d + W(u,v)$$

dopo la chiamata si ha:

$$v.d =$$

```
if v.d > u.d + W(u,v) then
  v.d ← u.d + W(u,v)
  v.π ← u
```

[Disuguaglianza triangolare 2]

$\forall (u,v) \in E$, allora $v.d \leq u.d + W(u,v)$ dopo la chiamata a **RELAX**(u, v, W)

Se prima della chiamata si ha:

$$v.d > u.d + W(u,v)$$

dopo la chiamata si ha:

$$v.d = u.d + W(u,v)$$

```
if v.d > u.d + W(u,v) then  
    v.d ← u.d + W(u,v)  
    v.π ← u
```

[Disuguaglianza triangolare 2]

$\forall (u,v) \in E$, allora $v.d \leq u.d + W(u,v)$ dopo la chiamata a **RELAX**(u, v, W)

Se prima della chiamata si ha:

$$v.d > u.d + W(u,v)$$

$$v.d \leq u.d + W(u,v)$$

dopo la chiamata si ha:

$$v.d = u.d + W(u,v)$$

```
if v.d > u.d + W(u,v) then  
    v.d ← u.d + W(u,v)  
    v.π ← u
```

[Disuguaglianza triangolare 2]

$\forall (u,v) \in E$, allora $v.d \leq u.d + W(u,v)$ dopo la chiamata a **RELAX**(u, v, W)

Se prima della chiamata si ha:

$$v.d > u.d + W(u,v)$$

$$v.d \leq u.d + W(u,v)$$

dopo la chiamata si ha:

$$v.d = u.d + W(u,v)$$

$$v.d \leq$$

```
if v.d > u.d + W(u,v) then  
    v.d ← u.d + W(u,v)  
    v.π ← u
```

[Disuguaglianza triangolare 2]

$\forall (u,v) \in E$, allora $v.d \leq u.d + W(u,v)$ dopo la chiamata a **RELAX**(u, v, W)

Se prima della chiamata si ha:

$$v.d > u.d + W(u,v)$$

$$v.d \leq u.d + W(u,v)$$

dopo la chiamata si ha:

$$v.d = u.d + W(u,v)$$

$$v.d \leq u.d + W(u,v)$$

```
if v.d > u.d + W(u,v) then  
    v.d ← u.d + W(u,v)  
    v.π ← u
```

[Disuguaglianza triangolare 2]

$\forall (u,v) \in E$, allora $v.d \leq u.d + W(u,v)$ dopo la chiamata a **RELAX**(u, v, W)

Se prima della chiamata si ha:

$$v.d > u.d + W(u,v)$$

$$v.d \leq u.d + W(u,v)$$

dopo la chiamata si ha:

$$v.d \leq u.d + W(u,v)$$

```
if v.d > u.d + W(u,v) then  
    v.d ← u.d + W(u,v)  
    v.π ← u
```

[Proprietà della convergenza]

Sia P_{sv} il cammino minimo da s a v e sia u il predecessore di v in P_{sv}

Si sta per chiamare **RELAX**(u, v, W).

Se $u.d = \delta(s, u)$, prima di **RELAX**(u, v, W), allora si avrà $v.d = \delta(s, v)$ dopo **RELAX**(u, v, W) e dopo eventuali passi di rilassamento successivi.



[Proprietà della convergenza]

Sia P_{sv} il cammino minimo da s a v e sia u il predecessore di v in P_{sv}

Si sta per chiamare **RELAX**(u, v, W).

Se $u.d = \delta(s, u)$, prima di **RELAX**(u, v, W), allora si avrà $v.d = \delta(s, v)$ dopo **RELAX**(u, v, W) e dopo eventuali passi di rilassamento successivi.

Si ha $u.d = \delta(s, u)$ dopo **RELAX**(u, v, W) - LIMITE SUPERIORE

Si ha $v.d \leq u.d + W(u, v)$ dopo **RELAX**(u, v, W) - DIS. TRIANGOLARE 2

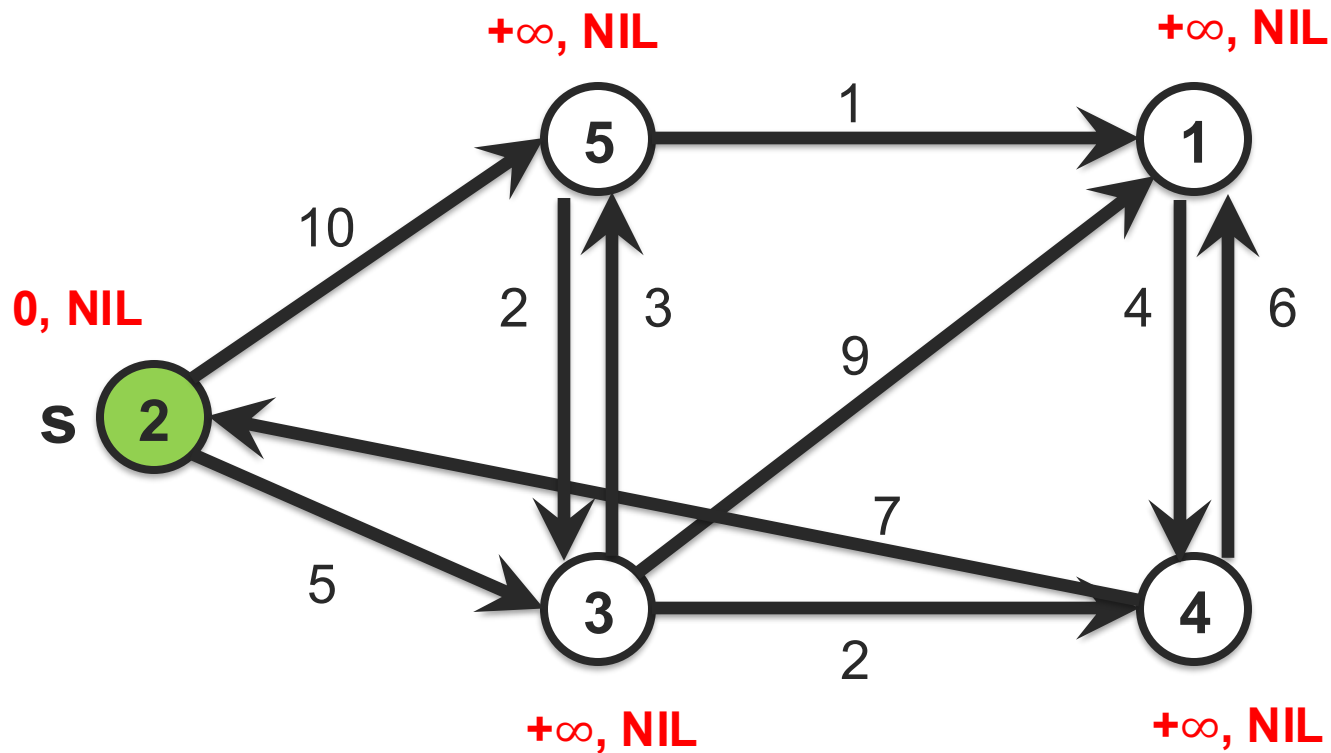
$\Rightarrow v.d \leq \delta(s, u) + W(u, v) = \delta(s, v) \Rightarrow v.d \leq \delta(s, v)$

Si ha poi $v.d \geq \delta(s, v)$ dopo **RELAX**(u, v, W) - LIMITE SUPERIORE

$\Rightarrow v.d = \delta(s, v)$ dopo **RELAX**(u, v, W) e dopo eventuali altri passi di rilassamento

[Algoritmo di Dijkstra]

1. Chiama **INITIALIZE-SINGLE-SOURCE** ($G = (V, E), s$)



[Algoritmo di Dijkstra]

1. Chiama **INITIALIZE-SINGLE-SOURCE** ($G = (V, E), s$)
2. $\forall (u, v) \in E$, chiama **RELAX** (u, v, W)

Al termine:

- ✓ $\forall x \in V, \mathbf{x.d} = \delta(s, x)$
- ✓ $\forall x \in V, \mathbf{x.\pi}$ è il predecessore di x in $\mathbf{P}_{sx}$

[Algoritmo di Dijkstra]

Prima di un'iterazione di rilassamento:

- ✓ **S**, insieme dei vertici x per cui si è arrivati a determinare $\delta(s,x)$ (cioè $\mathbf{x.d} = \delta(s,x)$)
- ✓ **V-S**, insieme dei vertici x per cui non si è ancora arrivati a determinare $\delta(s,x)$

Iterazione di rilassamento:

- viene estratto da **V-S** il vertice u con il minimo valore dell'attributo $\mathbf{d}$.
Si dimostra che:
 - $\Rightarrow \mathbf{u.d} = \delta(s,u)$
 - $\Rightarrow \mathbf{u.\pi}$ è predecessore di u in $\mathbf{P_{su}}$
- per ogni $v \in \text{adj}(u)$, viene eseguita **RELAX(u, v, W)**

$$\begin{aligned} \mathbf{V-S} = \emptyset &\Rightarrow \forall x \in V, \mathbf{x.d} = \delta(s,x) \\ &\Rightarrow \forall x \in V, \mathbf{x.\pi} \text{ è predecessore di } x \text{ in } \mathbf{P_{sx}} \end{aligned}$$

[Algoritmo di Dijkstra]

- Inizialmente $\mathbf{V-S} = V$, $\mathbf{S} = \emptyset$
- La sorgente s è il primo vertice estratto da $\mathbf{V-S}$
- $\mathbf{V-S}$ viene implementato con una *min-priority queue* Q

[Algoritmo di Dijkstra]

Procedura DIJKSTRA (V, E, W, s)

$S \leftarrow$ empty set

$Q \leftarrow$ empty min_priority queue //V-S

INITIALIZE-SINGLE-SOURCE ($G = (V, E), s$)

foreach $v \in V$ **do**

 INSERT(Q, v)

while $Q \neq \emptyset$ **do**

$u \leftarrow$ EXTRACT-MIN(Q)

$S \leftarrow S \cup \{u\}$

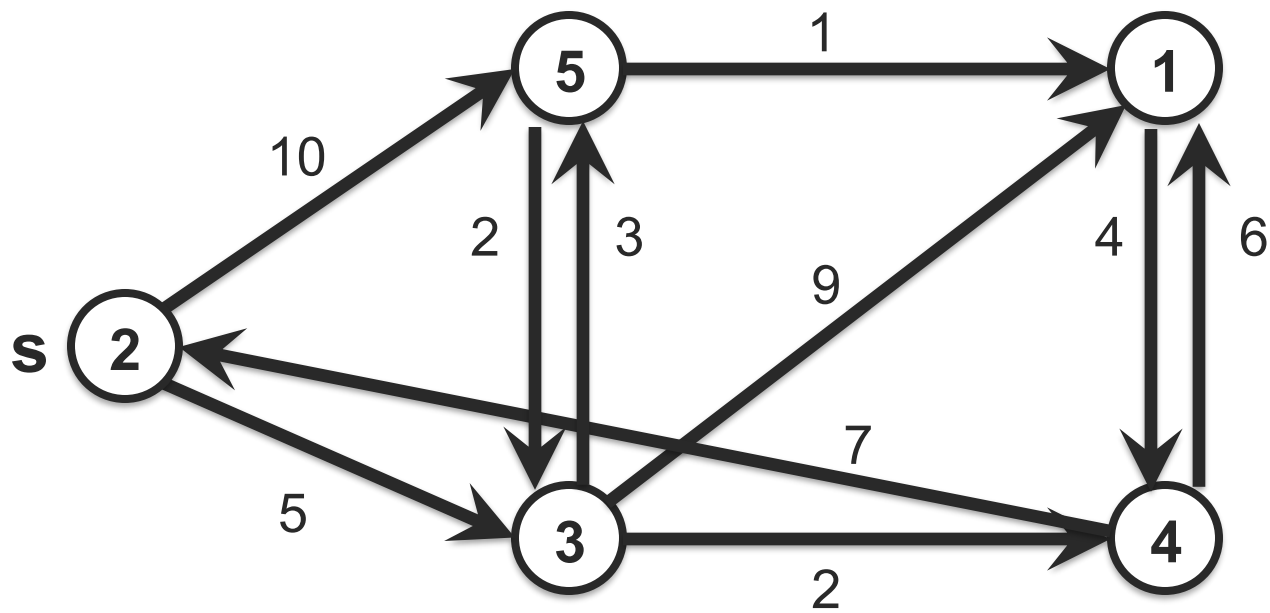
foreach $v \in \text{adj}(u)$ **do**

 RELAX(u, v, W)

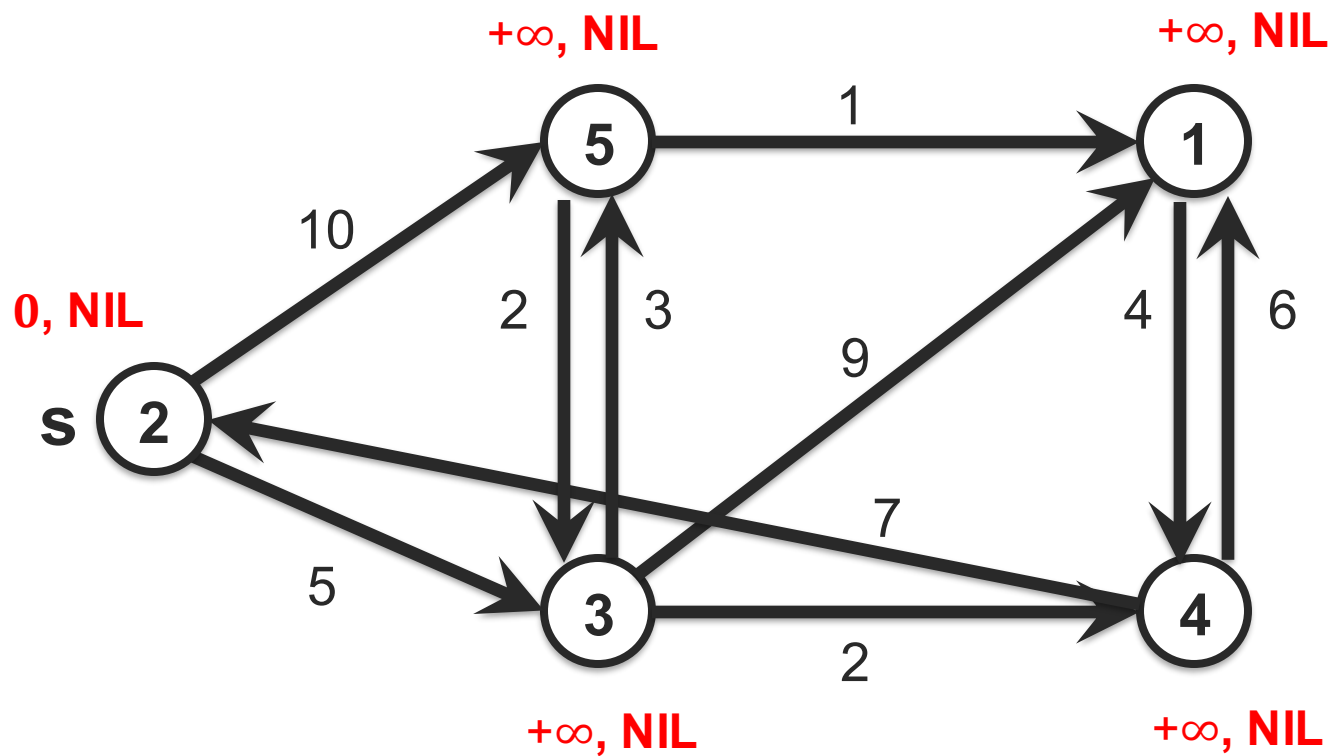
if $v.d$ è diminuito **then**

 DECREASE-KEY($Q, v, v.d$)

[Esempio]

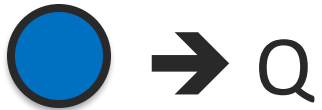
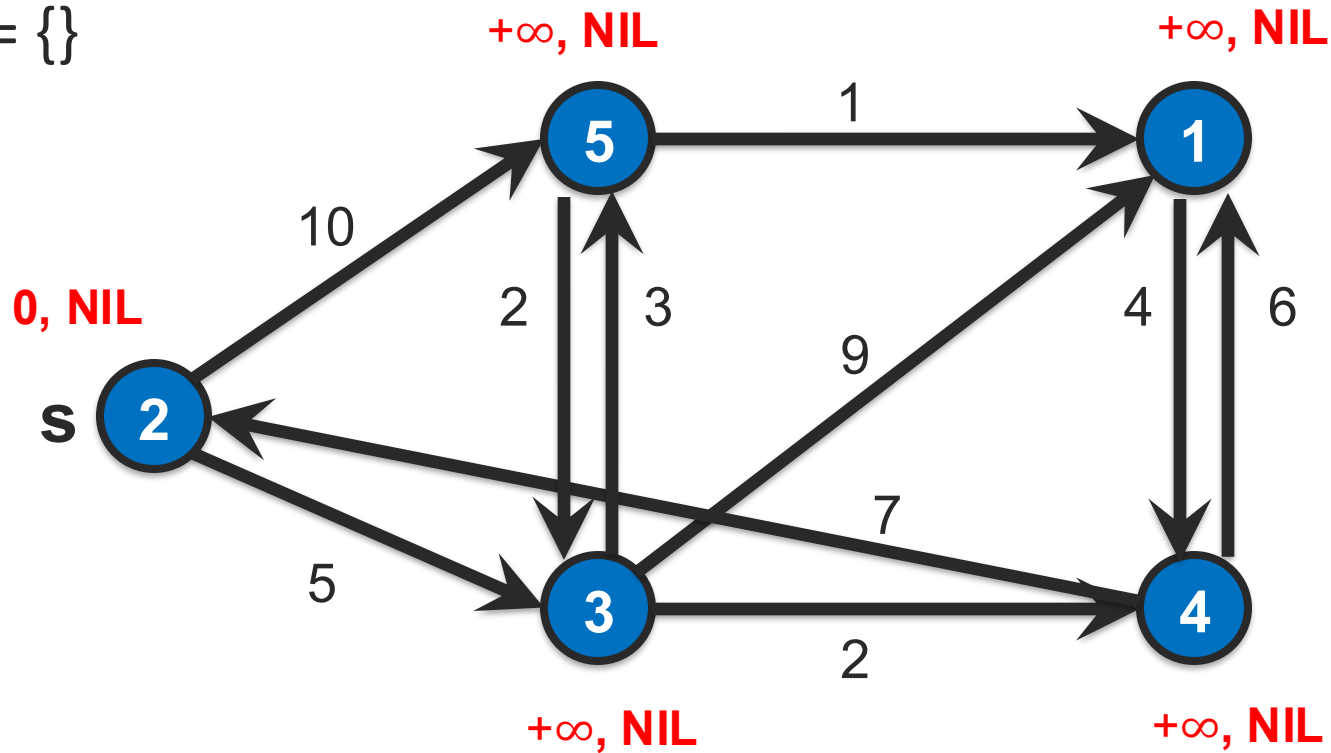


[Esempio]



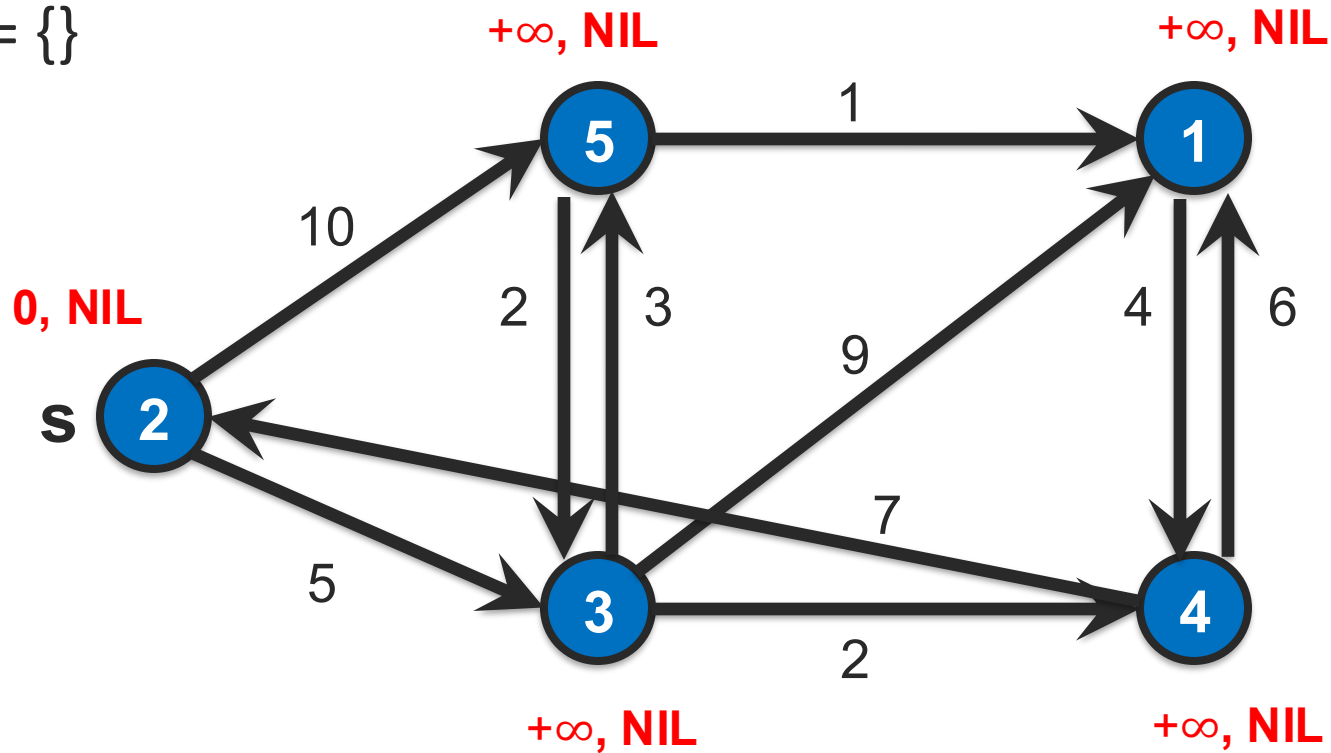
[Esempio]

$S = \{ \}$



[Esempio]

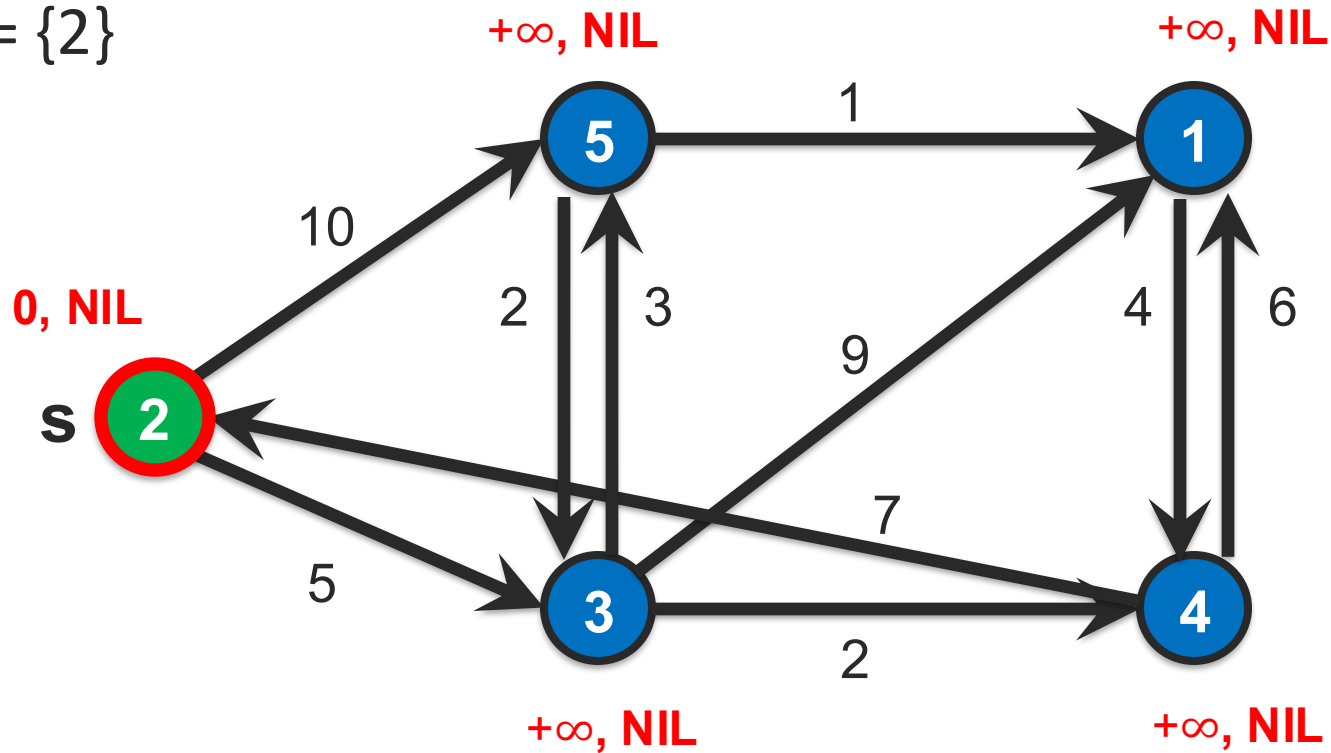
$S = \{\}$



Estrazione da Q del vertice ?

[Esempio]

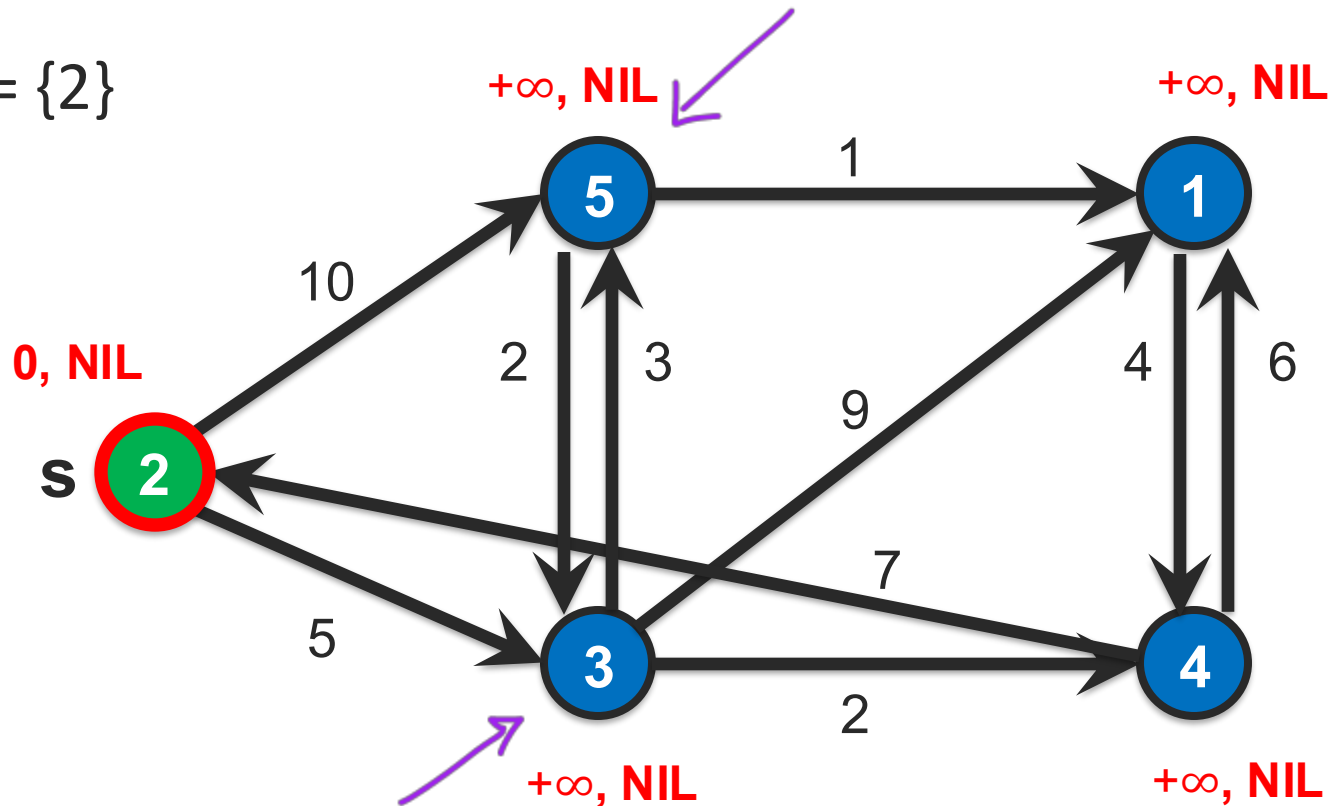
$S = \{2\}$



Estrazione da Q del vertice 2

[Esempio]

$S = \{2\}$

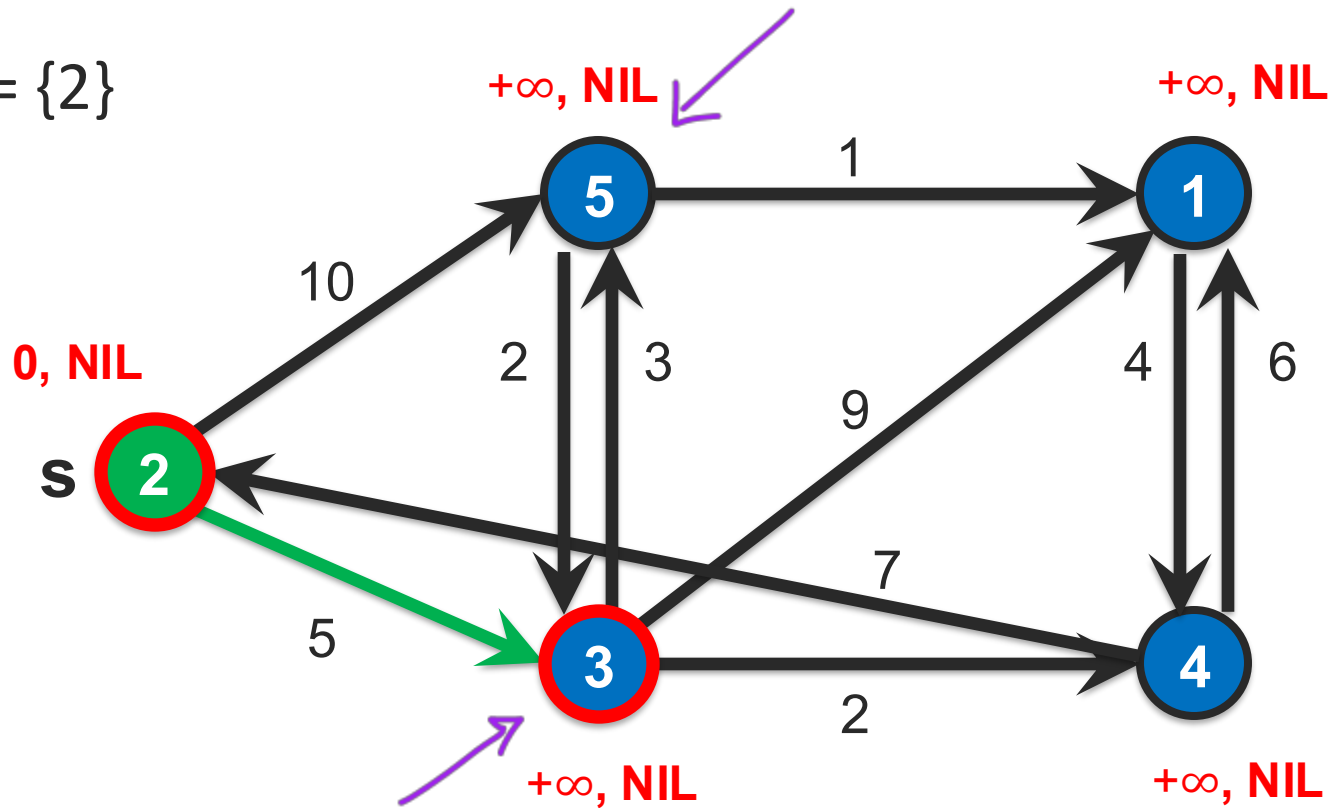


Estrazione da Q del vertice 2

Adiacenti $\rightarrow 3, 5$

[Esempio]

$S = \{2\}$

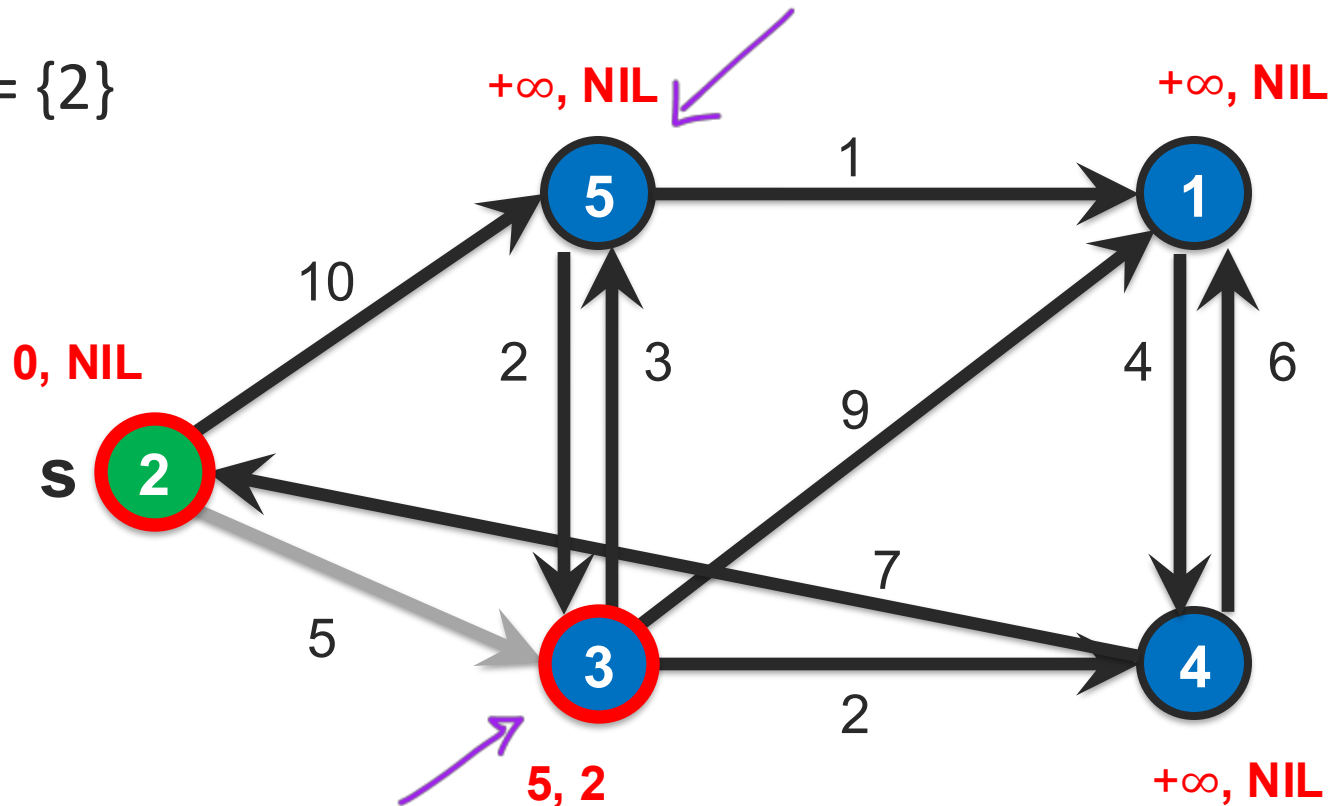


Estrazione da Q del vertice 2

Adiacenti $\rightarrow$ 3, 5

[Esempio]

$S = \{2\}$

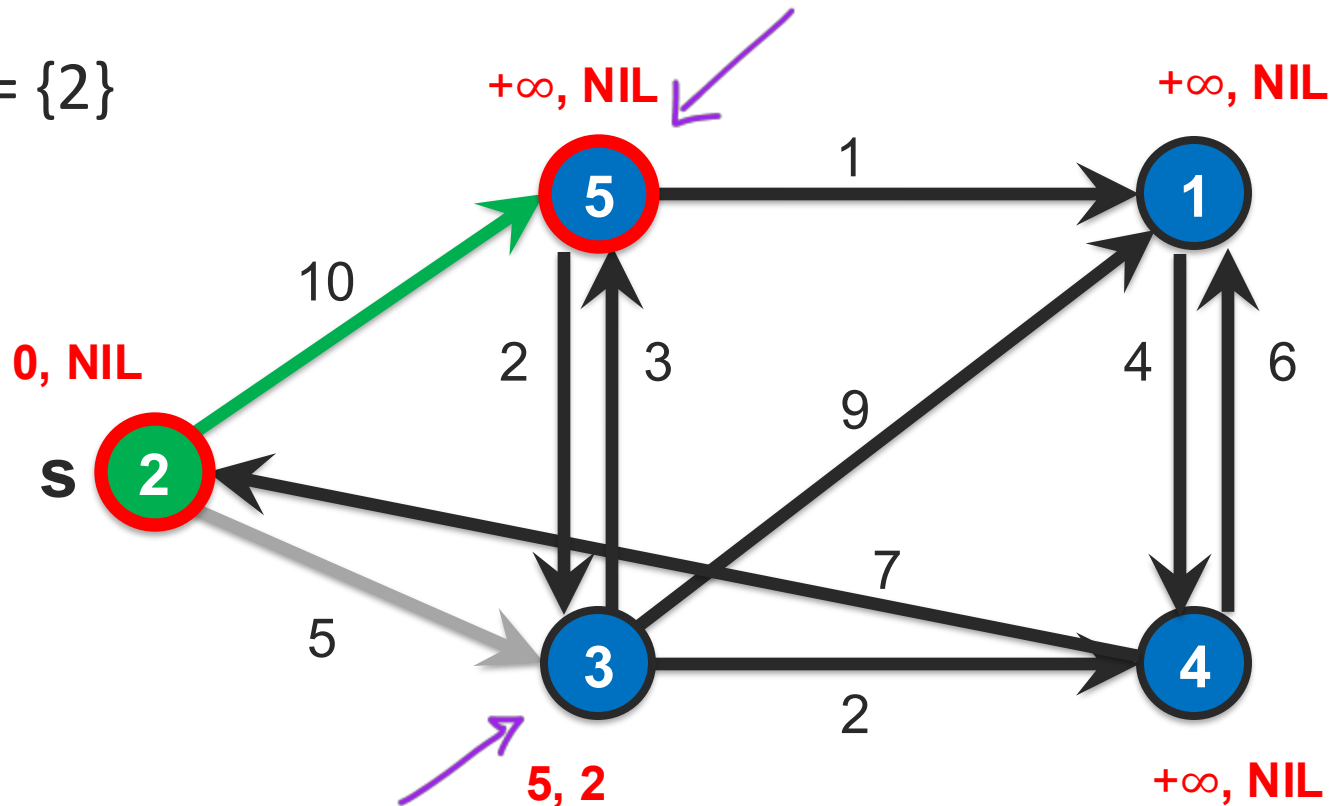


Estrazione da Q del vertice 2

Adiacenti $\rightarrow$ 3, 5

[Esempio]

$S = \{2\}$

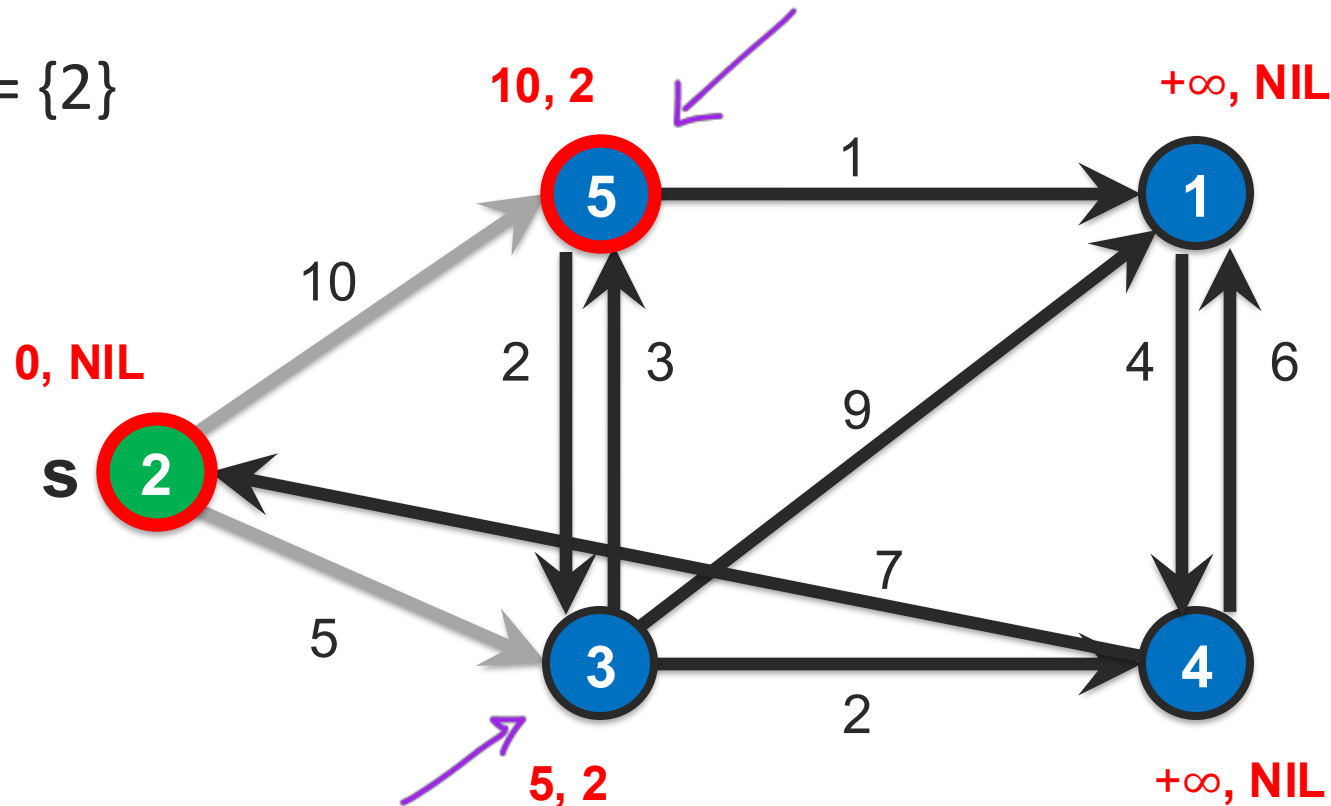


Estrazione da Q del vertice 2

Adiacenti $\rightarrow 3, \underline{5}$

[Esempio]

$S = \{2\}$

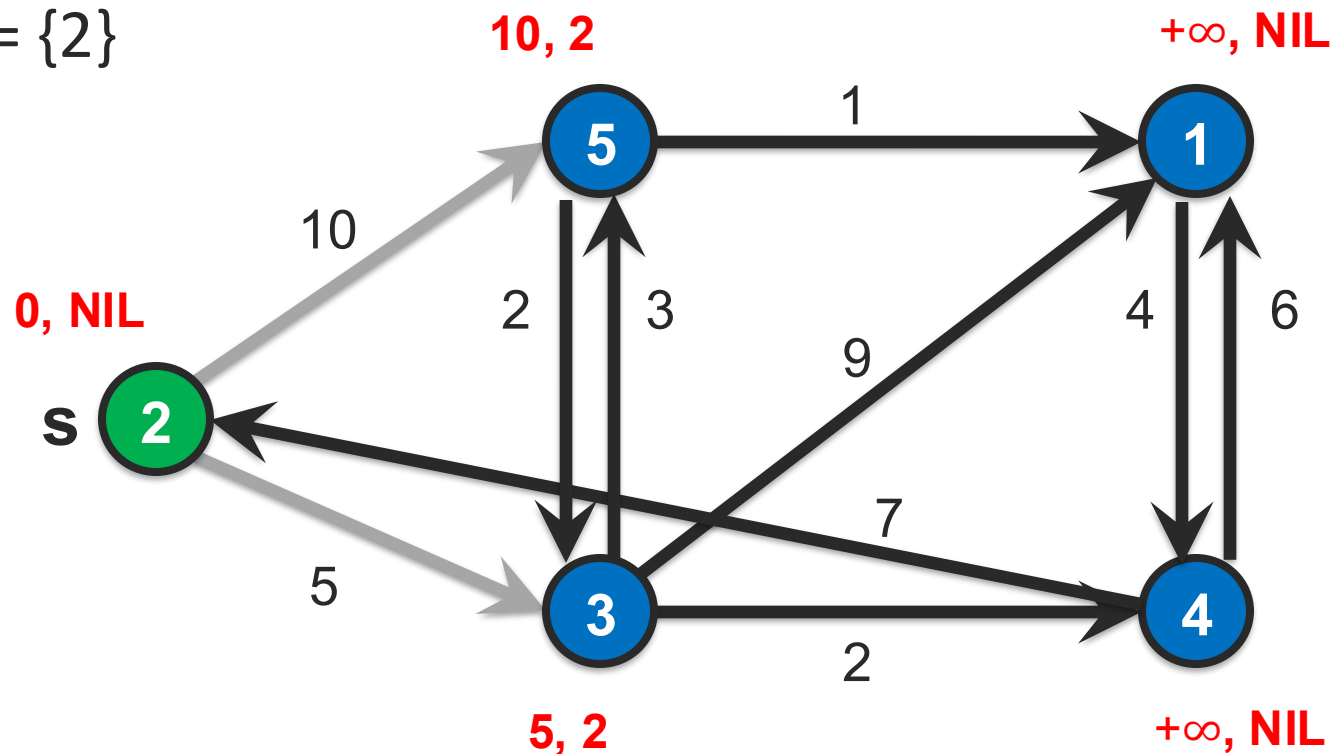


Estrazione da Q del vertice 2

Adiacenti $\rightarrow 3, \underline{5}$

[Esempio]

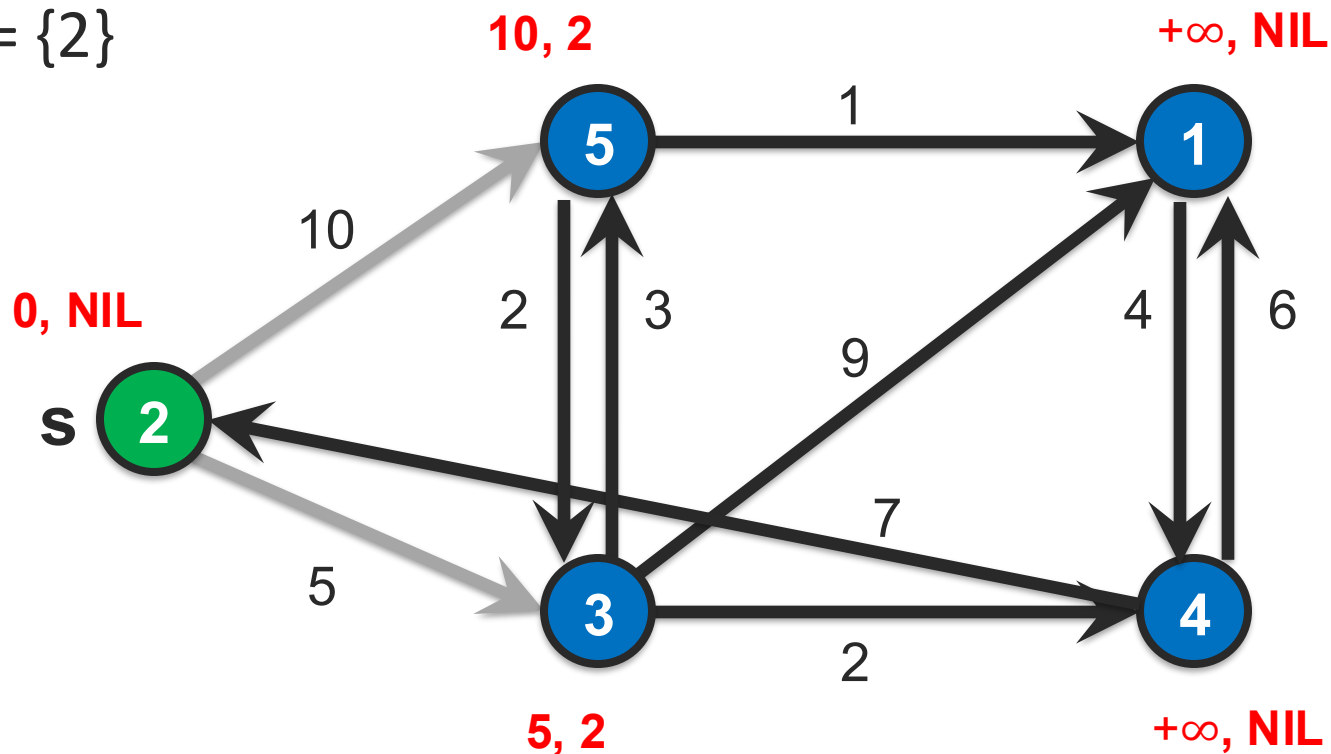
$S = \{2\}$



Estrazione da Q del vertice 2

[Esempio]

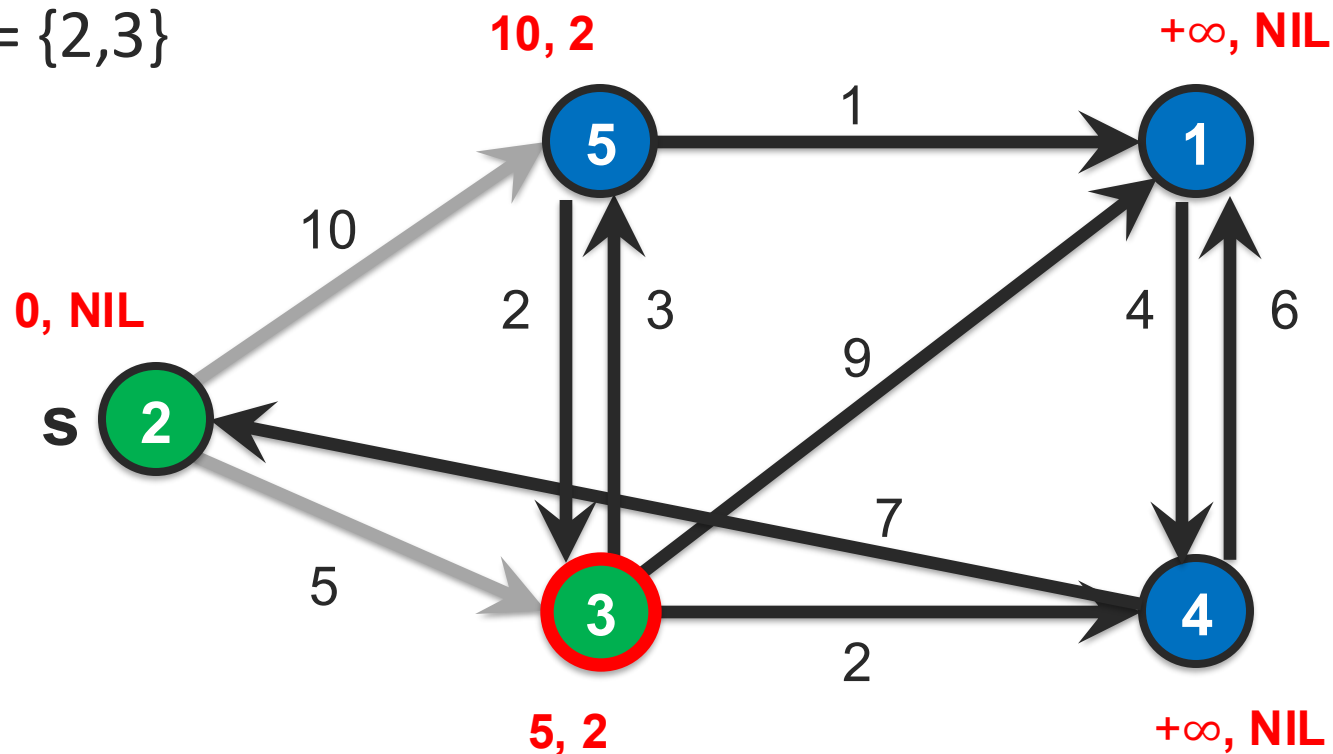
$S = \{2\}$



Estrazione da Q del vertice ?

[Esempio]

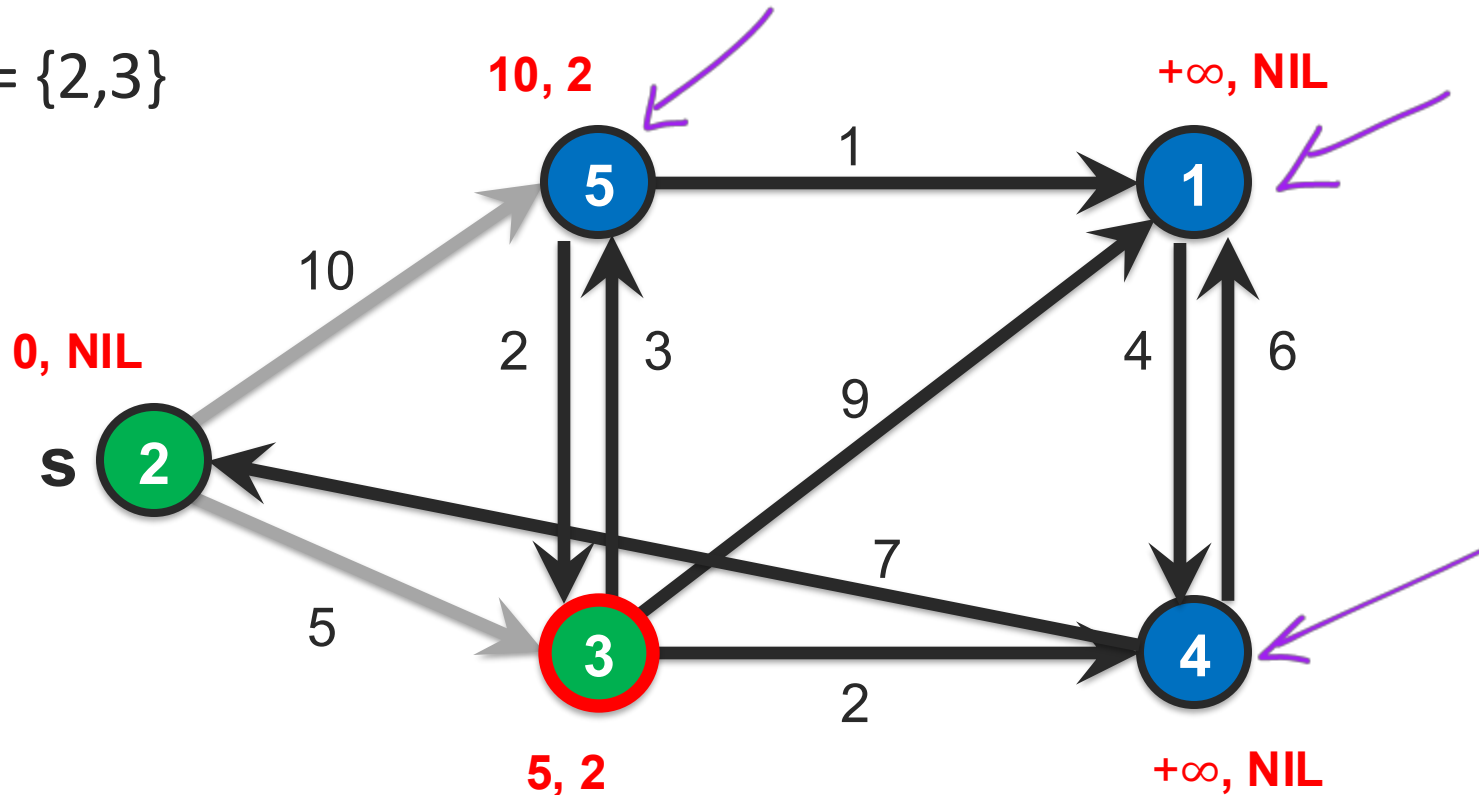
$S = \{2, 3\}$



Estrazione da Q del vertice 3

[Esempio]

$S = \{2, 3\}$

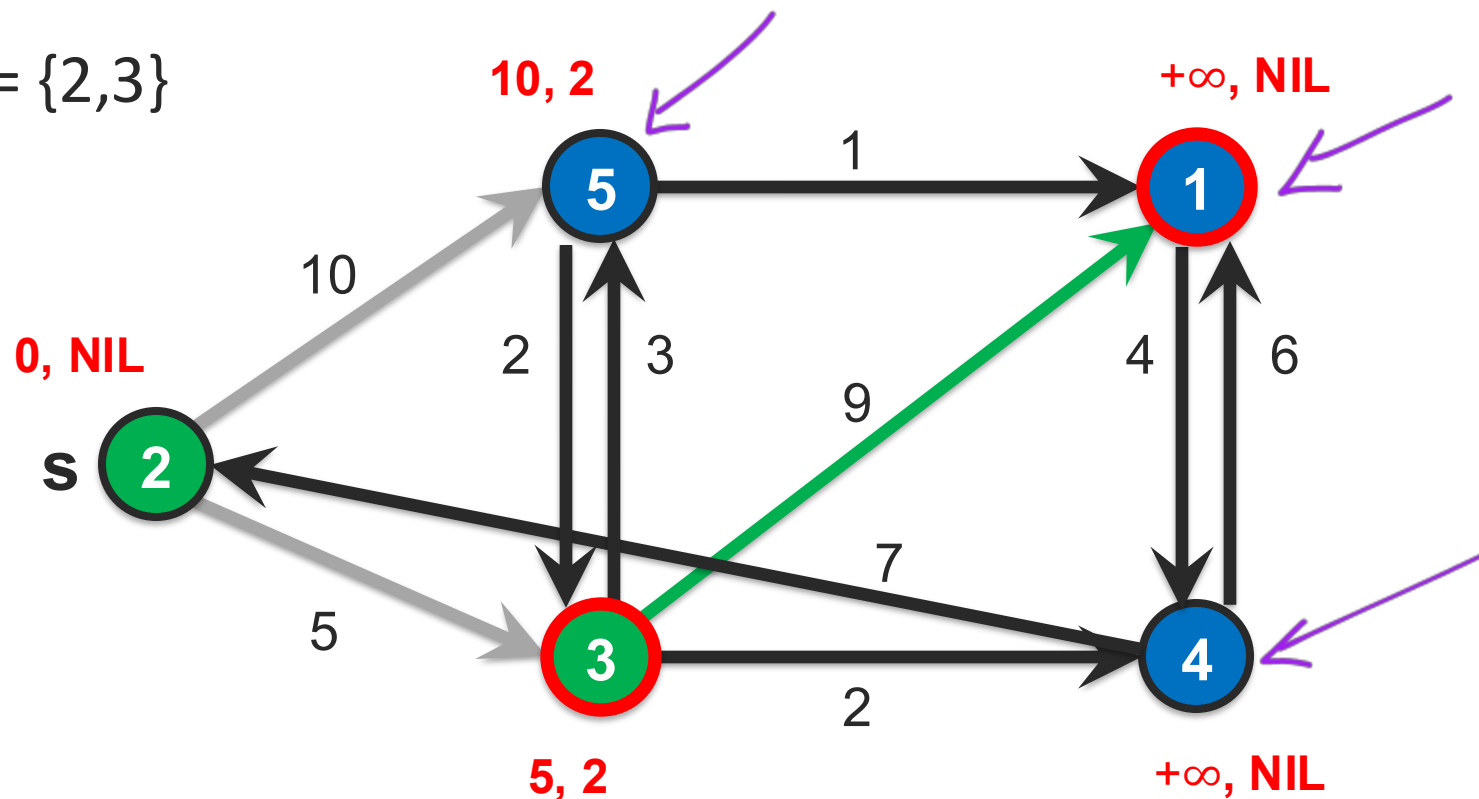


Estrazione da Q del vertice 3

Adiacenti $\rightarrow 1, 4, 5$

[Esempio]

$S = \{2, 3\}$

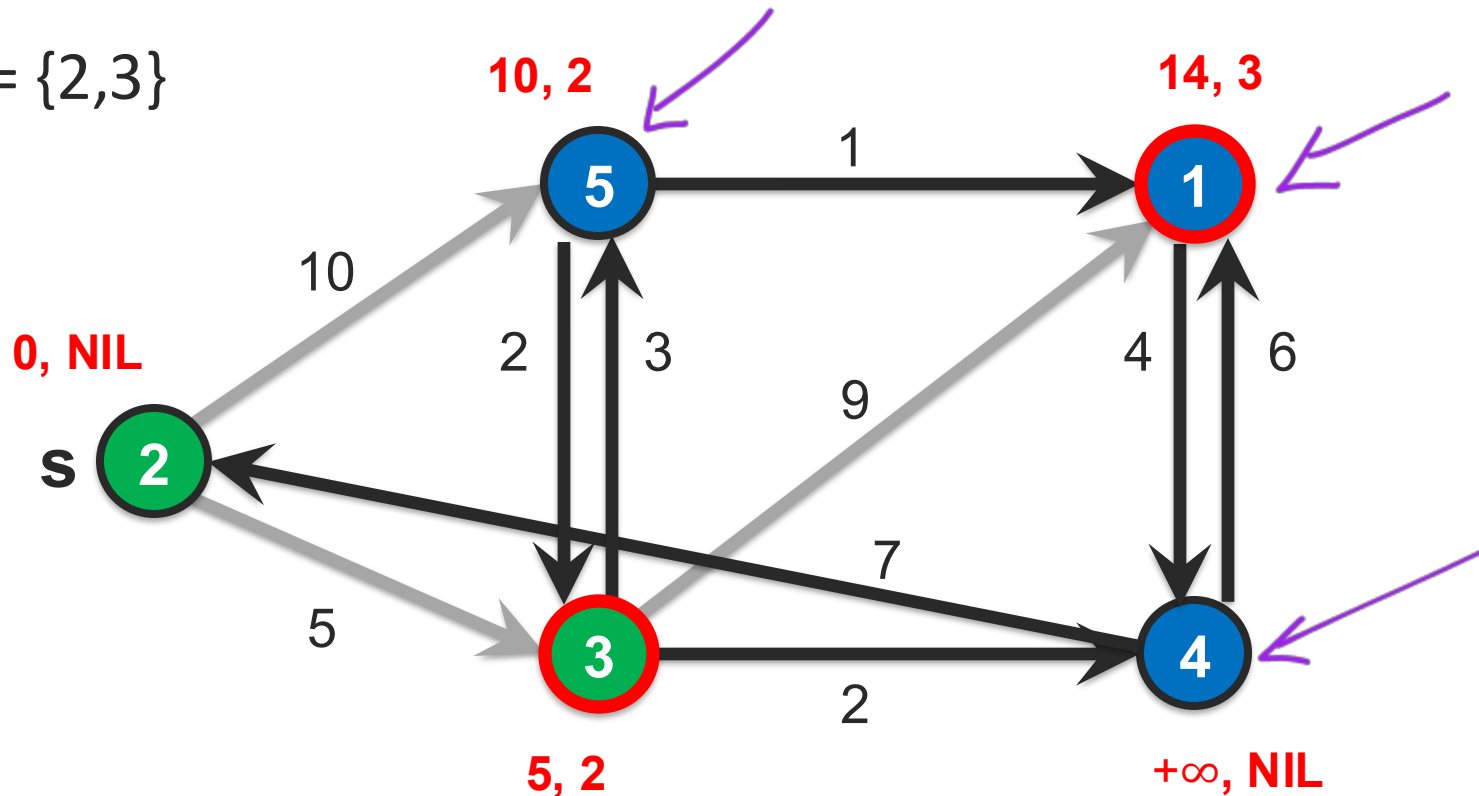


Estrazione da Q del vertice 3

Adiacenti $\rightarrow$ 1, 4, 5

[Esempio]

$S = \{2, 3\}$

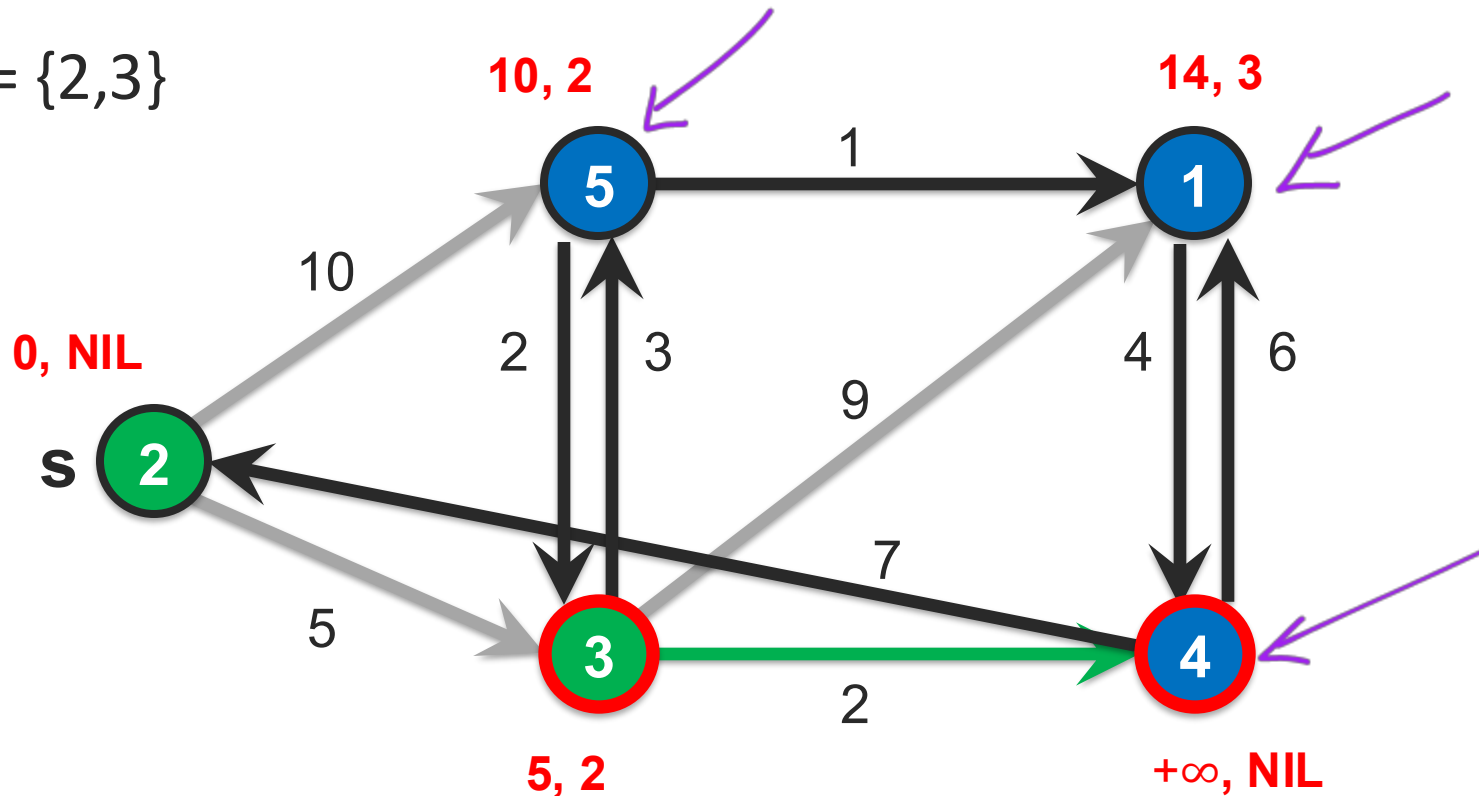


Estrazione da Q del vertice 3

Adiacenti → 1, 4, 5

[Esempio]

$S = \{2, 3\}$

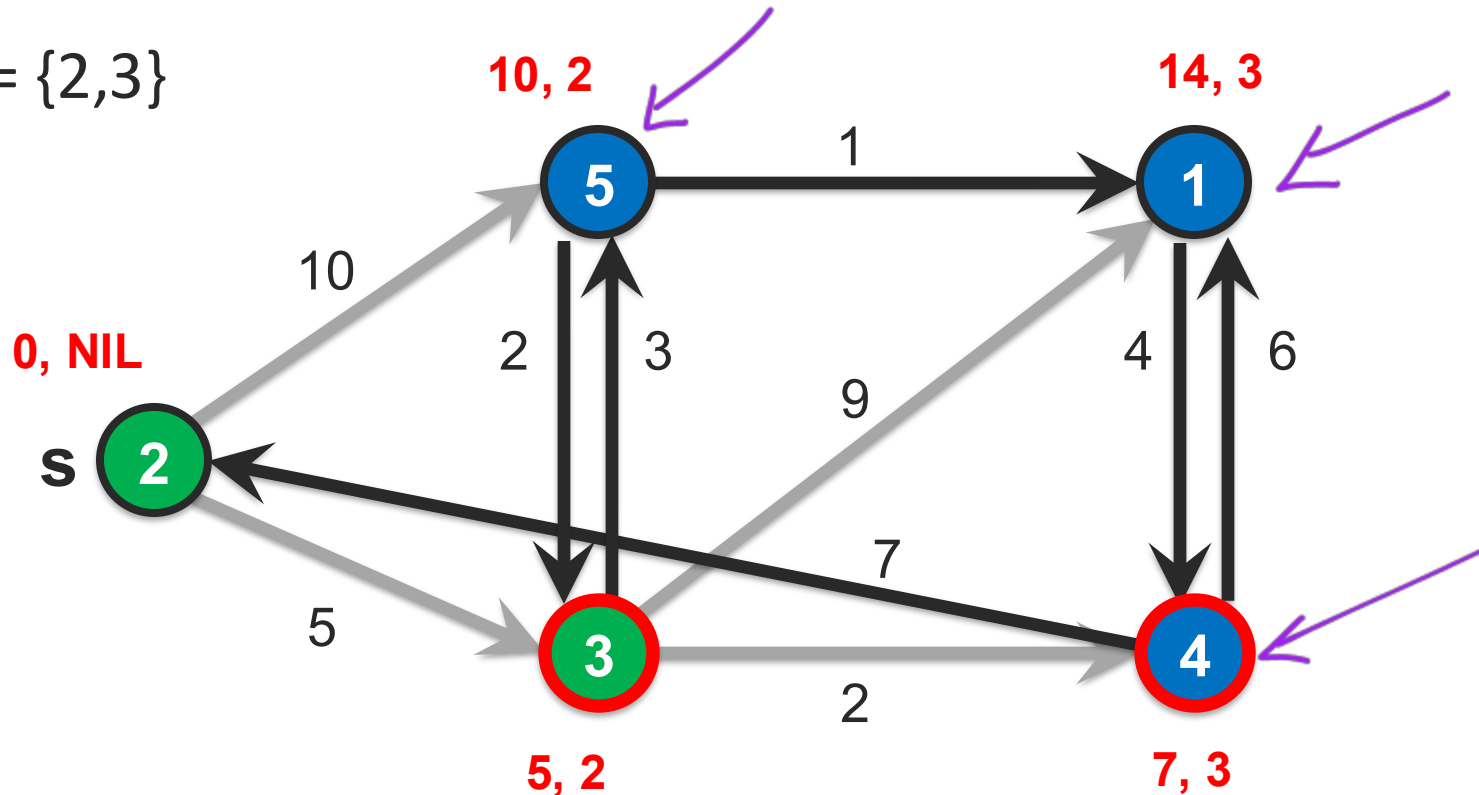


Estrazione da Q del vertice 3

Adiacenti $\rightarrow 1, \underline{4}, 5$

[Esempio]

$S = \{2, 3\}$

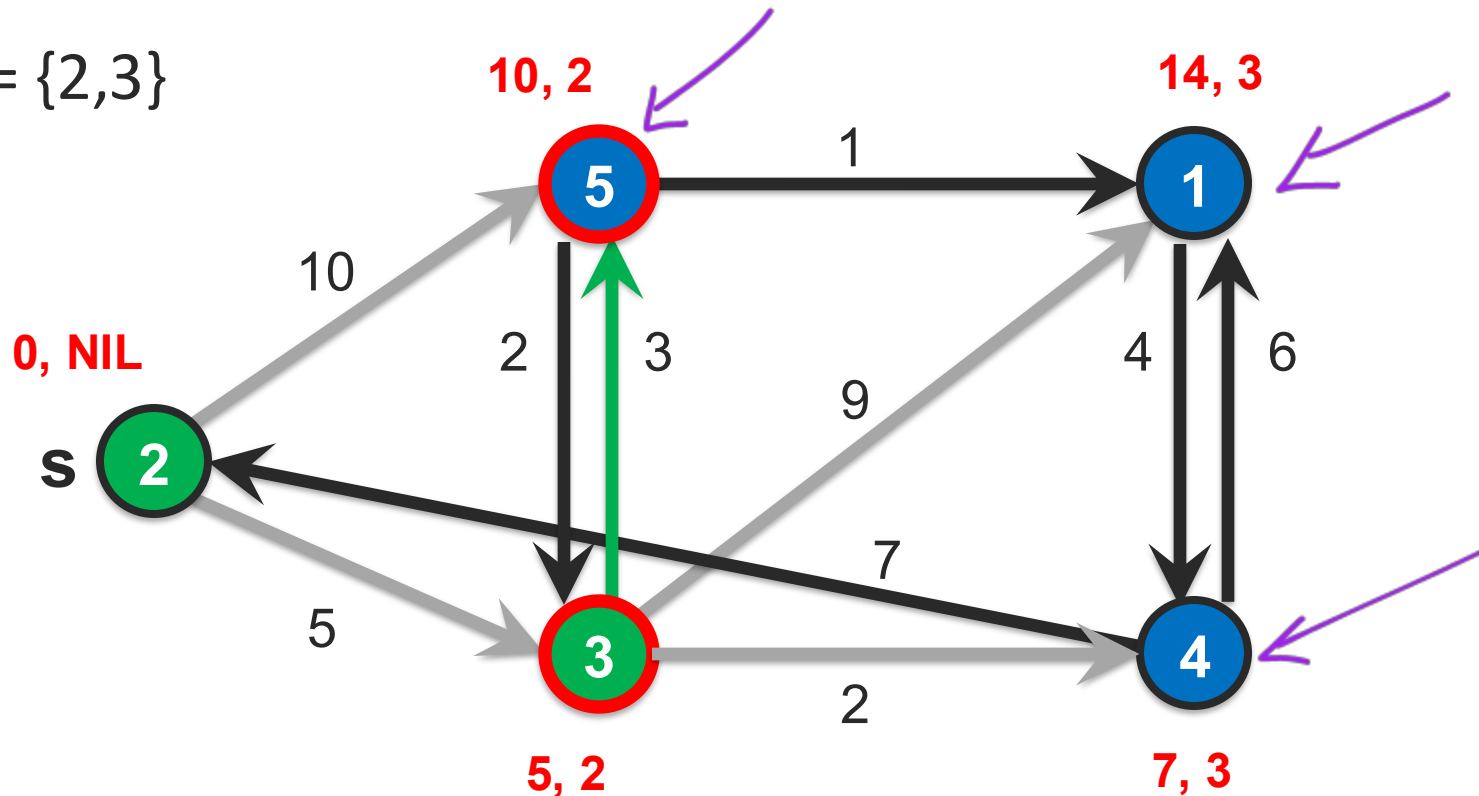


Estrazione da Q del vertice 3

Adiacenti → 1, 4, 5

[Esempio]

$S = \{2, 3\}$

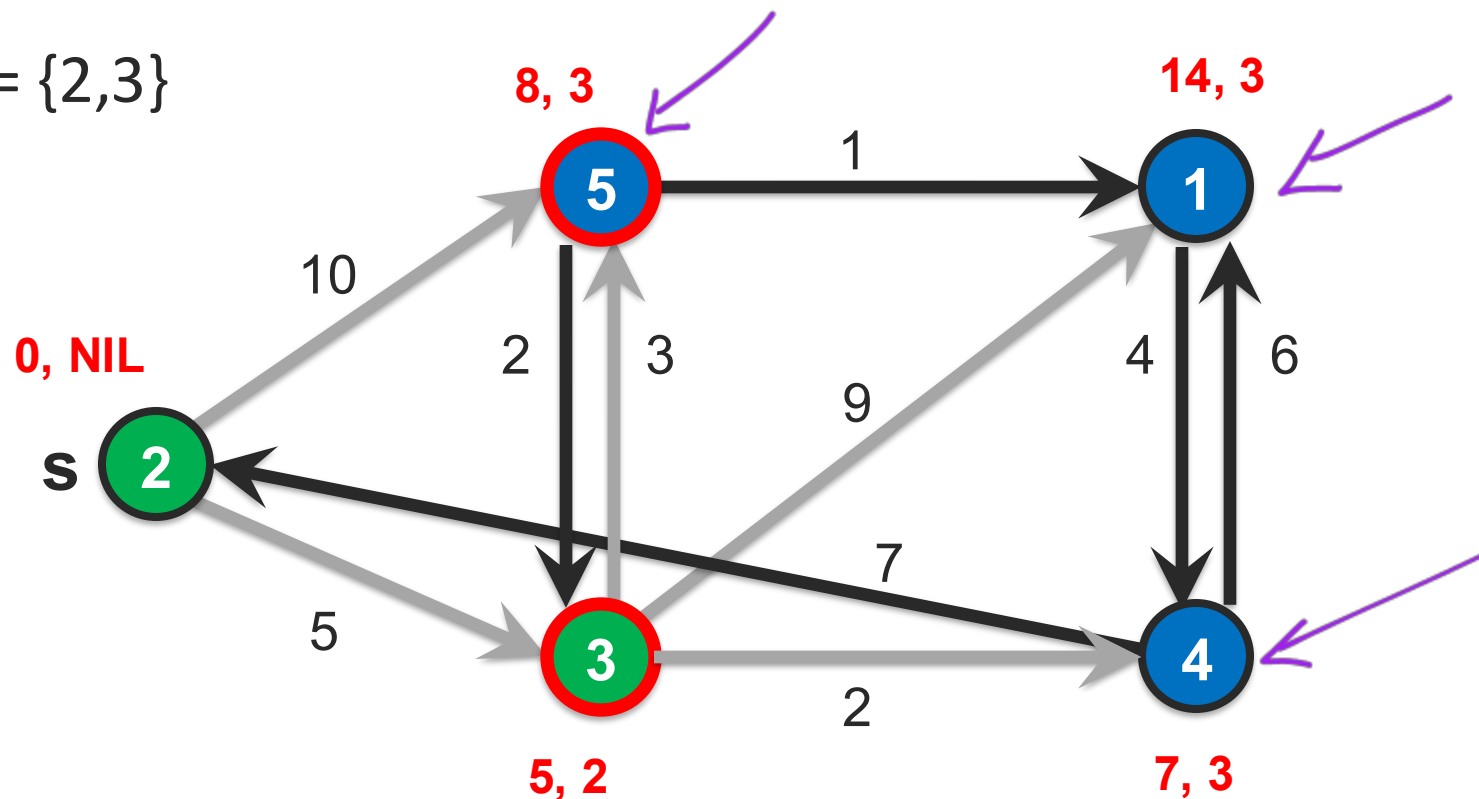


Estrazione da Q del vertice 3

Adiacenti → 1, 4, 5

[Esempio]

$S = \{2, 3\}$

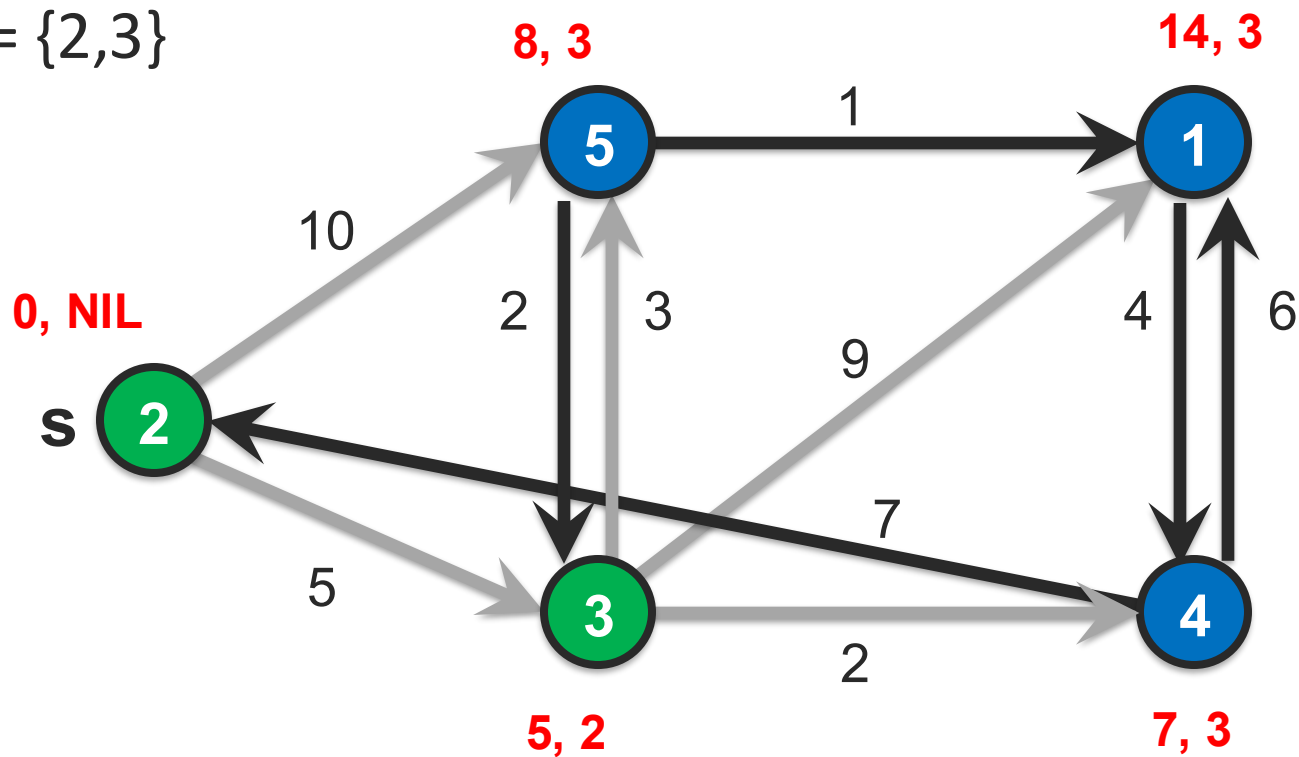


Estrazione da Q del vertice 3

Adiacenti → 1, 4, 5

[Esempio]

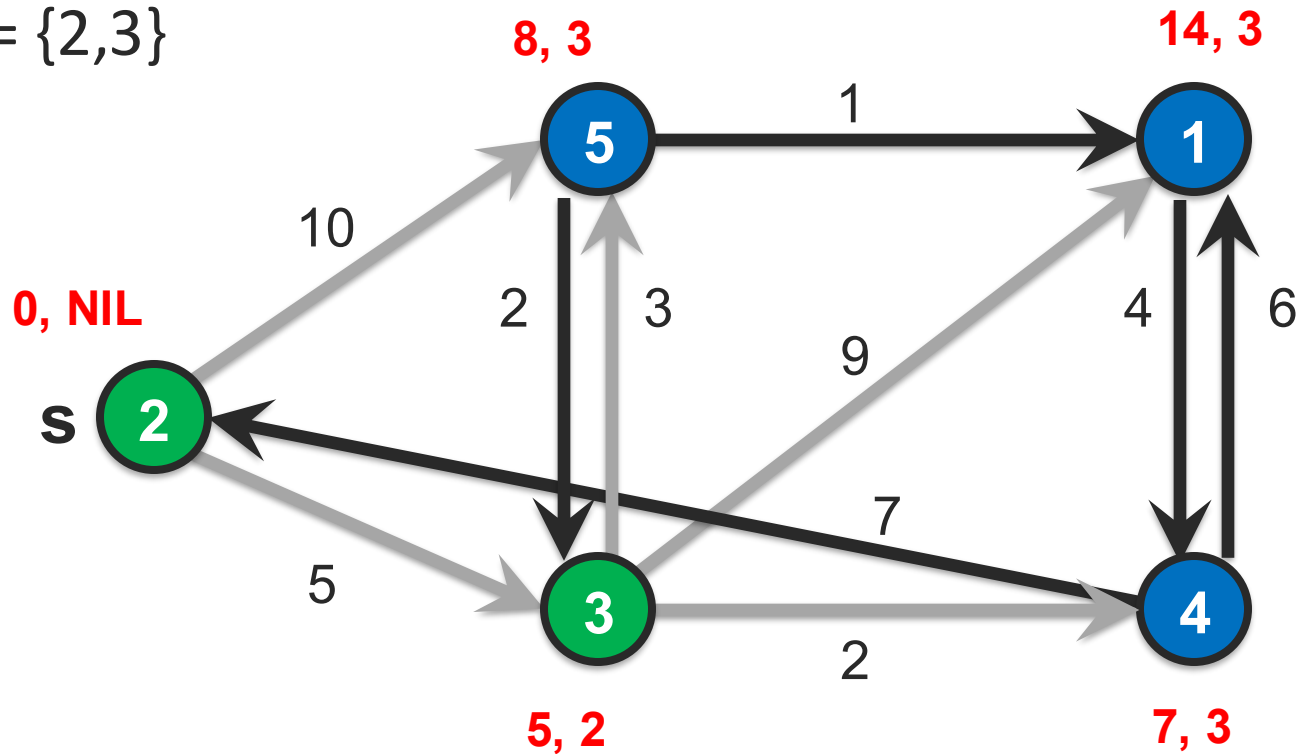
$S = \{2, 3\}$



Estrazione da Q del vertice 3

[Esempio]

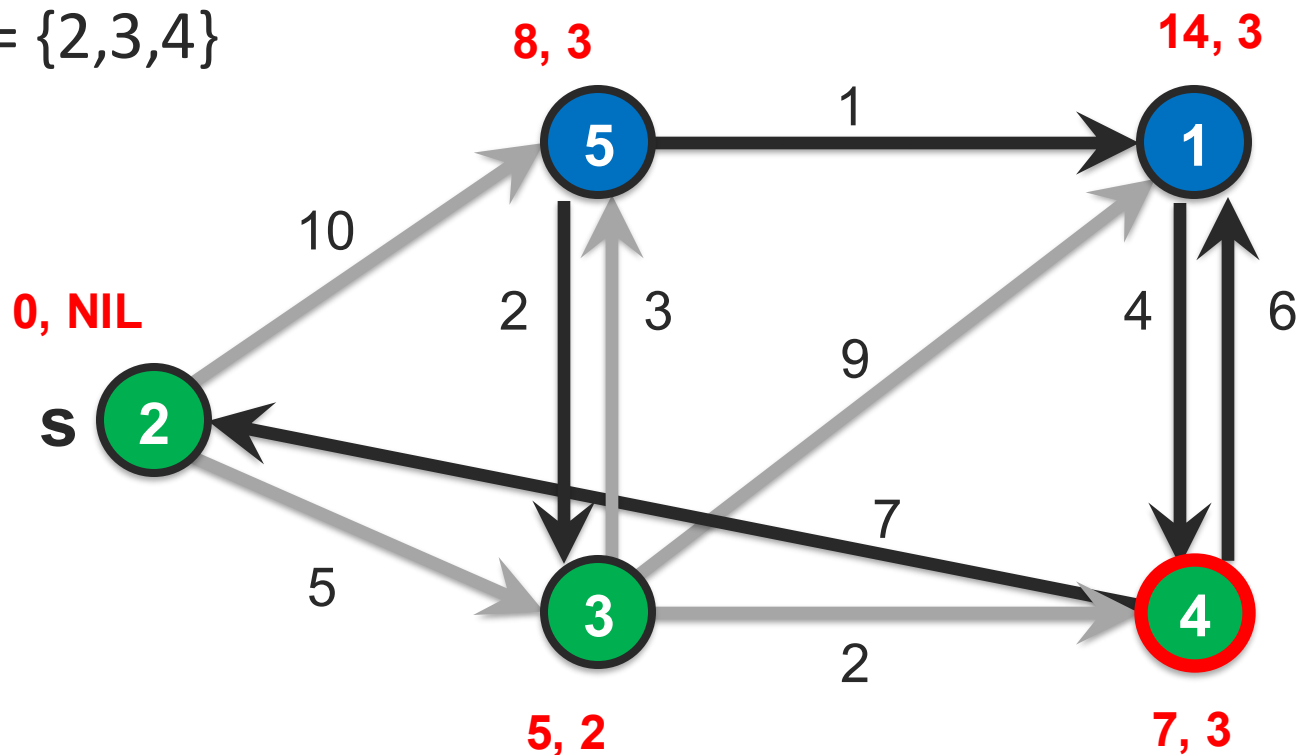
$S = \{2, 3\}$



Estrazione da Q del vertice ?

[Esempio]

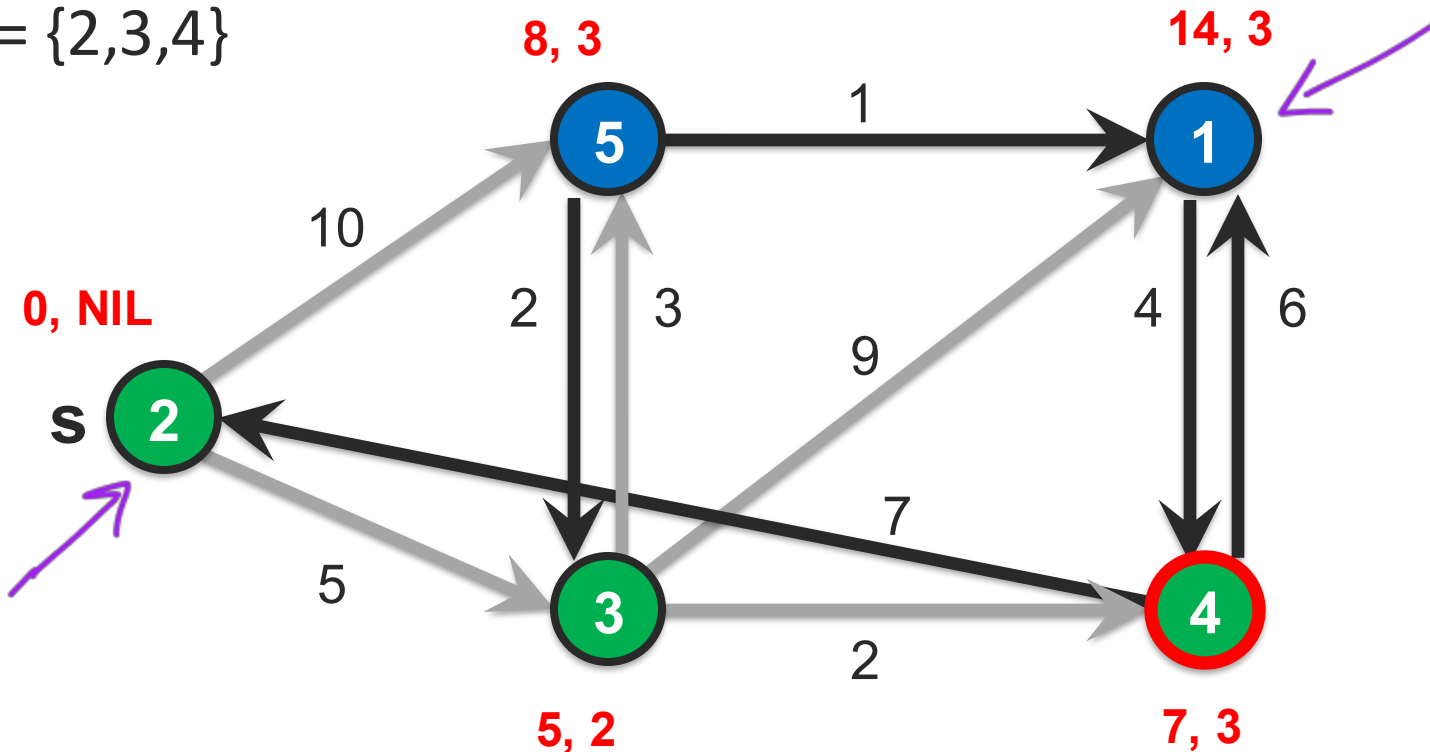
$S = \{2, 3, 4\}$



Estrazione da Q del vertice 4

[Esempio]

$S = \{2, 3, 4\}$

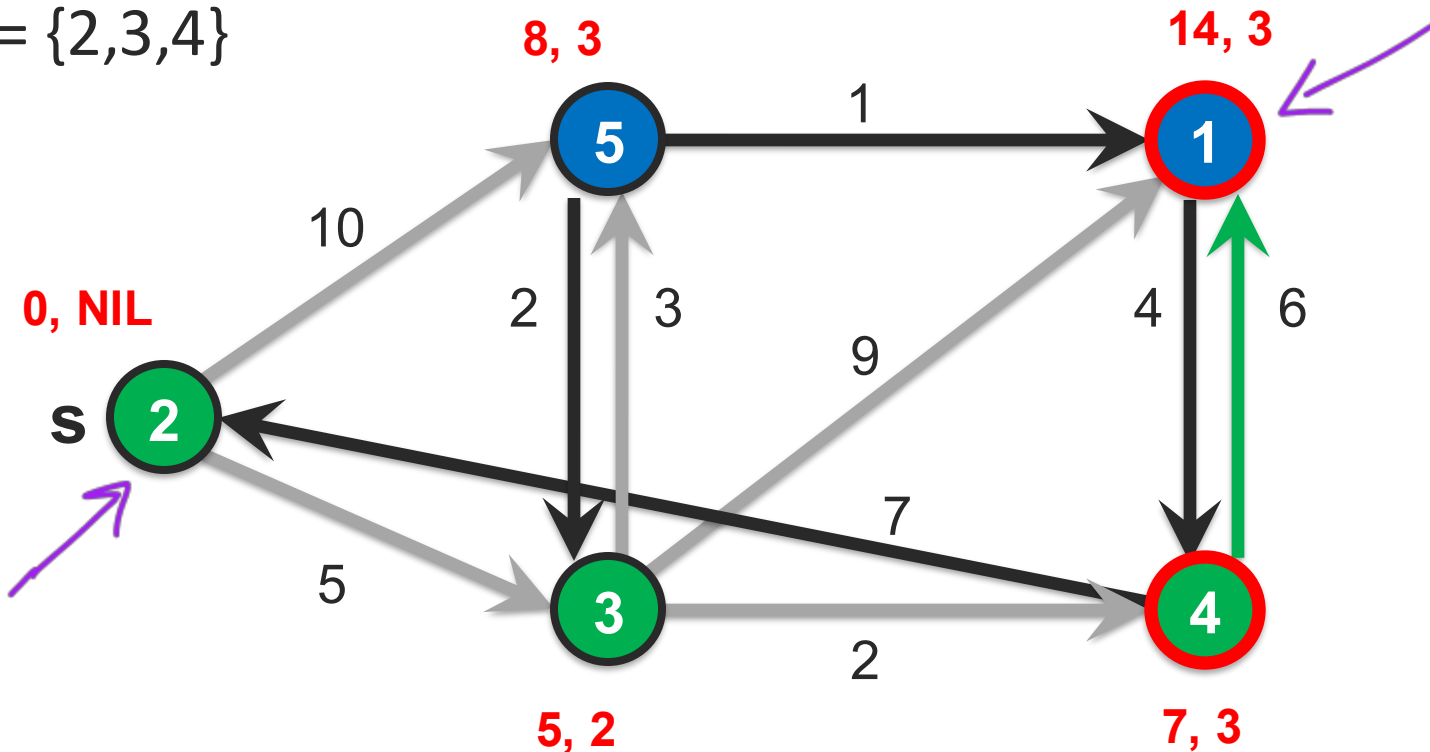


Estrazione da Q del vertice 4

Adiacenti $\rightarrow$ 1, 2

[Esempio]

$S = \{2, 3, 4\}$

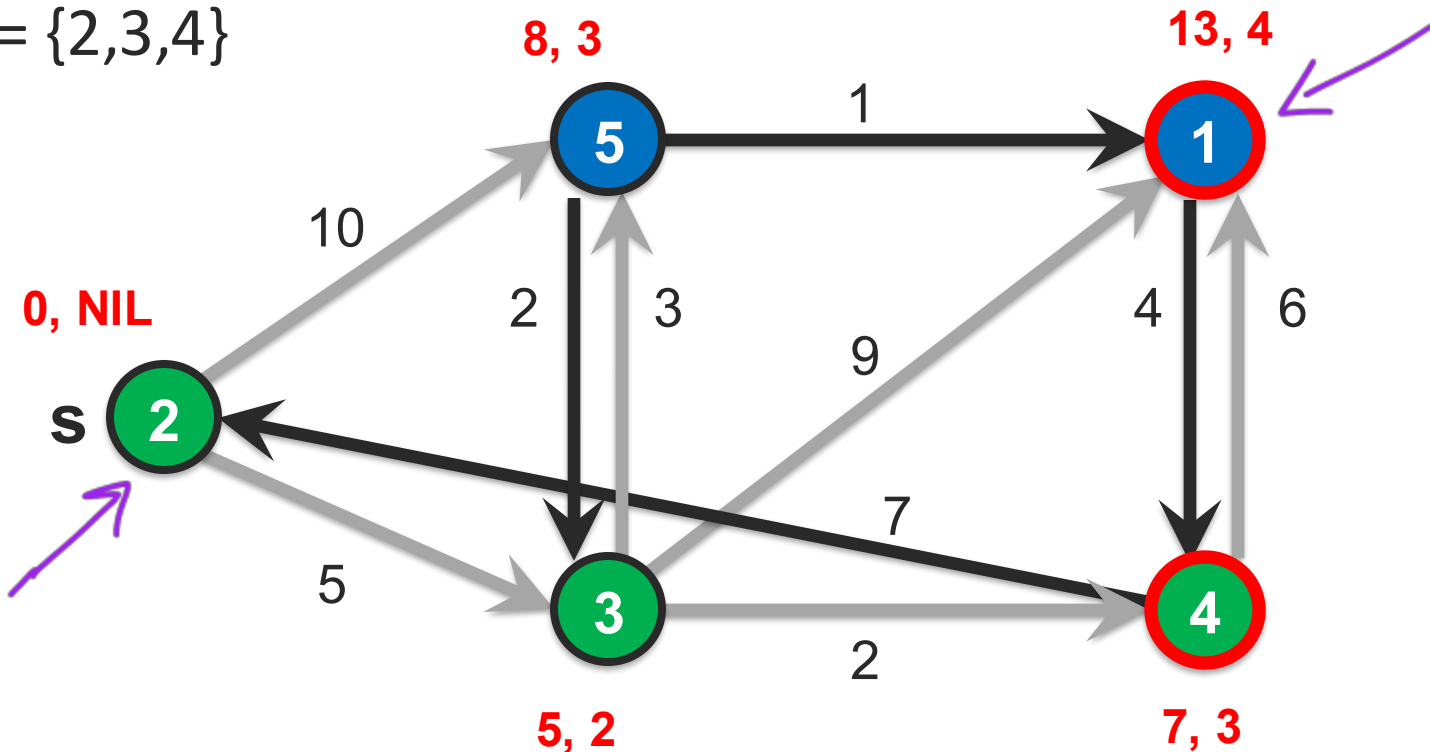


Estrazione da Q del vertice 4

Adiacenti → 1, 2

[Esempio]

$S = \{2, 3, 4\}$

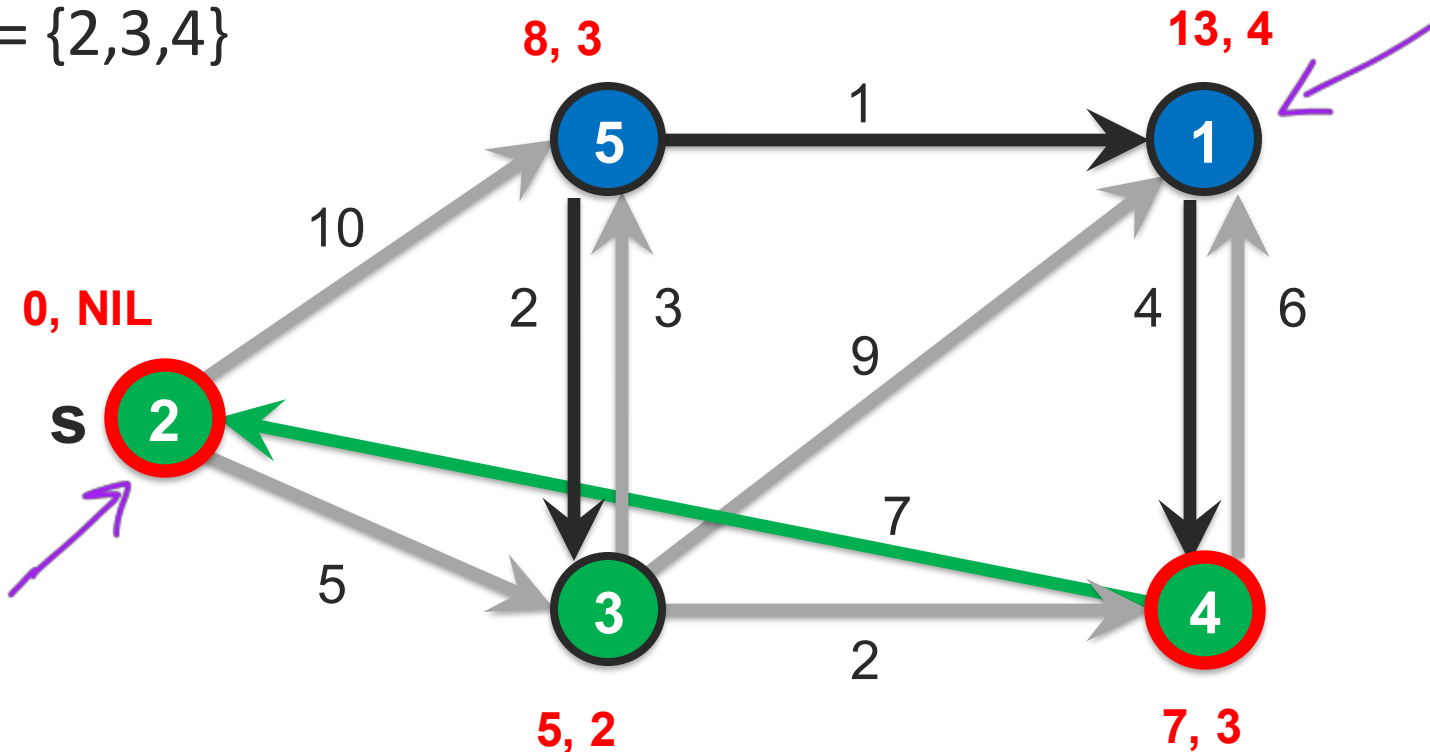


Estrazione da Q del vertice 4

Adiacenti $\rightarrow$ 1, 2

[Esempio]

$S = \{2, 3, 4\}$

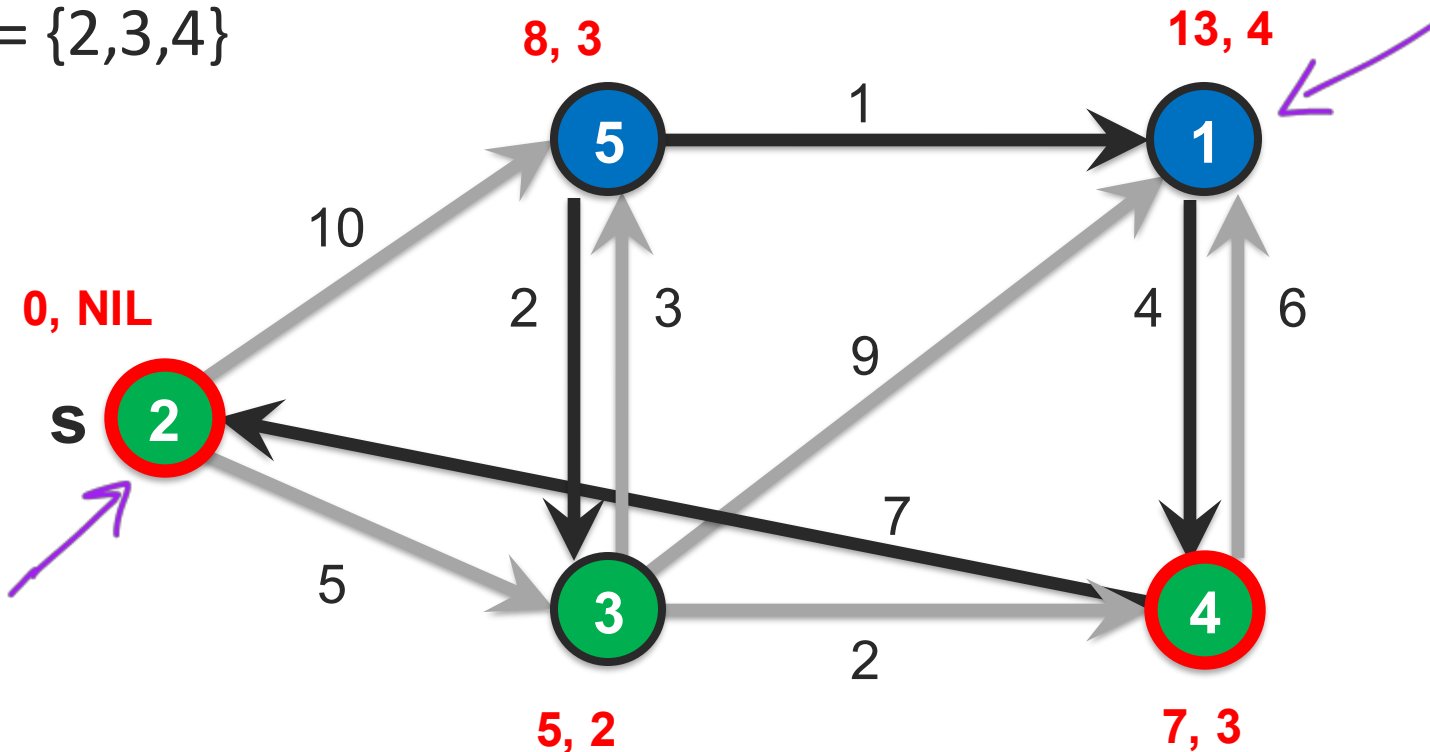


Estrazione da Q del vertice 4

Adiacenti → 1, 2

[Esempio]

$S = \{2, 3, 4\}$

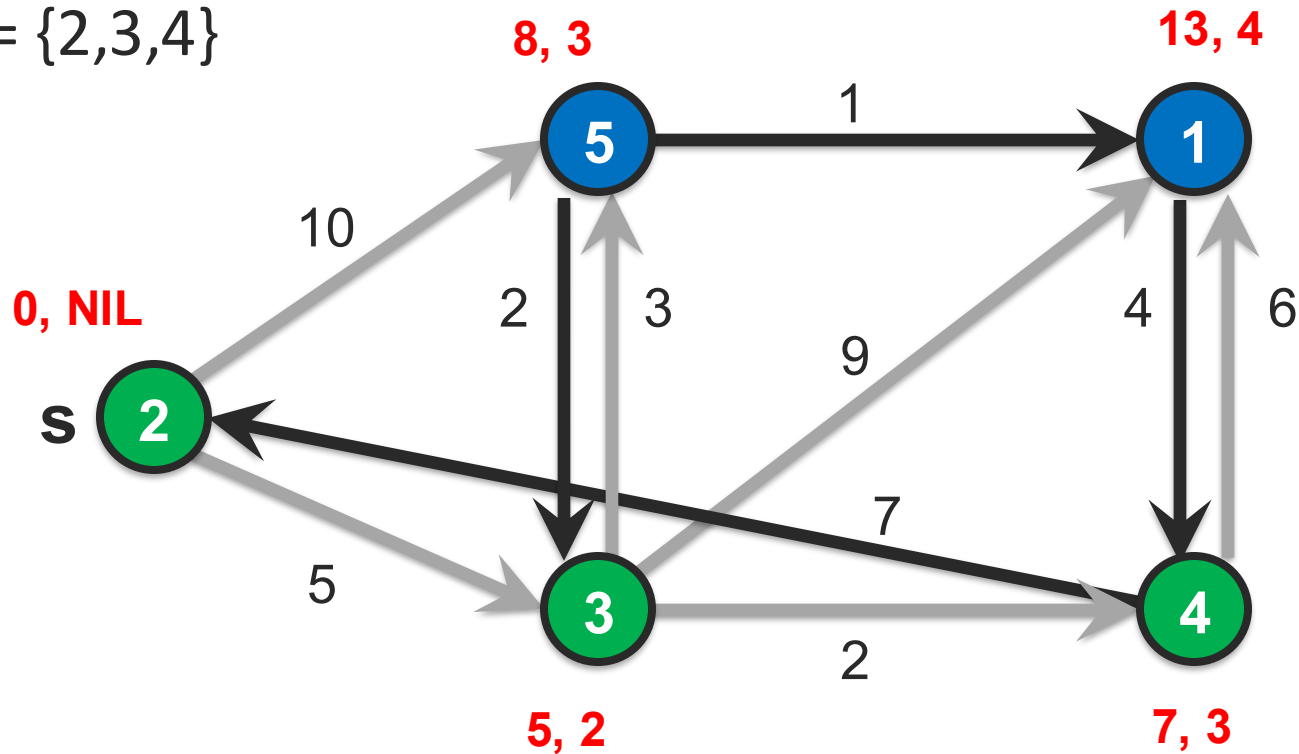


Estrazione da Q del vertice 4

Adiacenti → 1, 2

[Esempio]

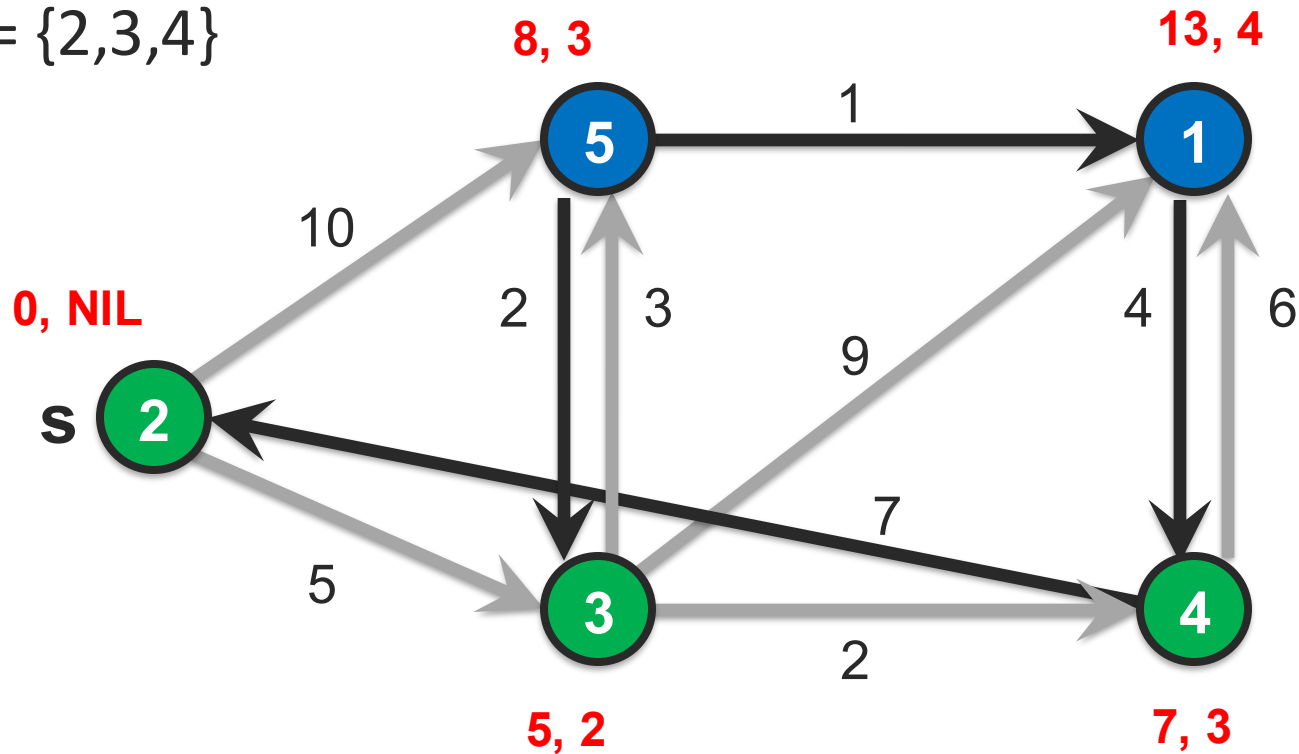
$S = \{2, 3, 4\}$



Estrazione da Q del vertice 4

[Esempio]

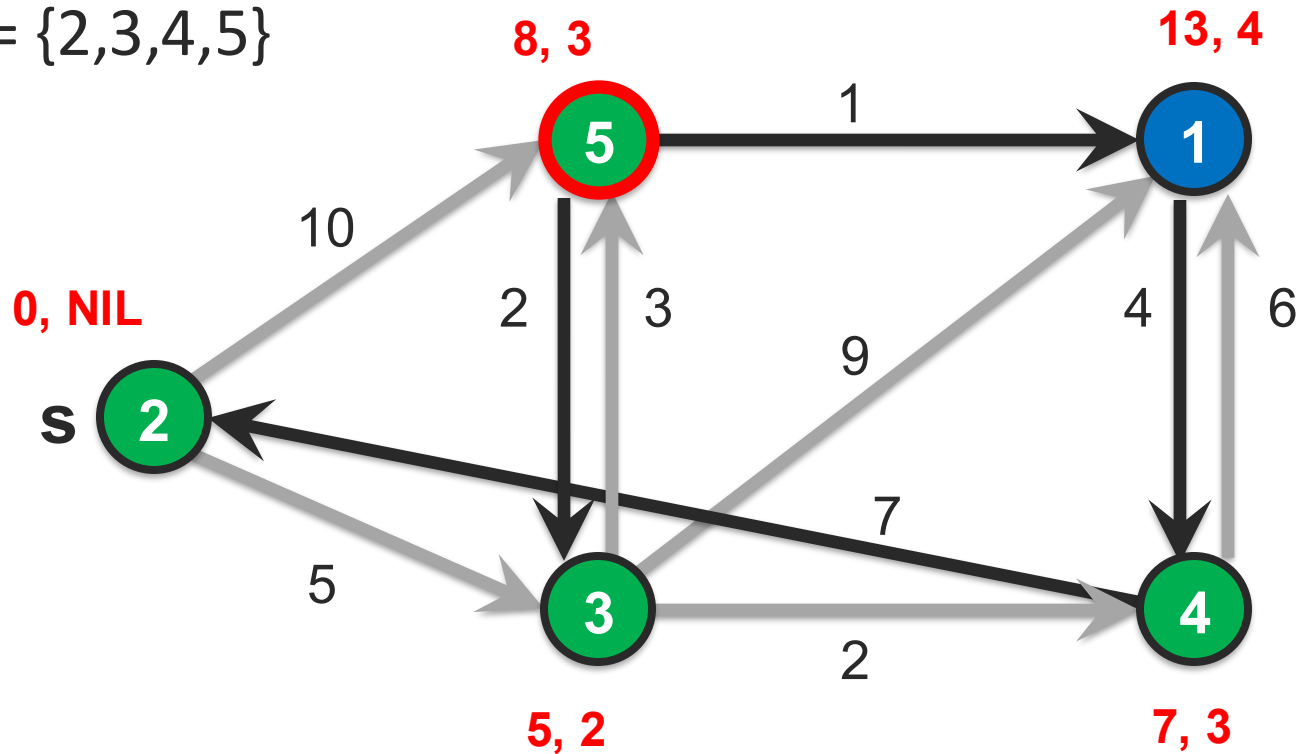
$S = \{2, 3, 4\}$



Estrazione da Q del vertice ?

[Esempio]

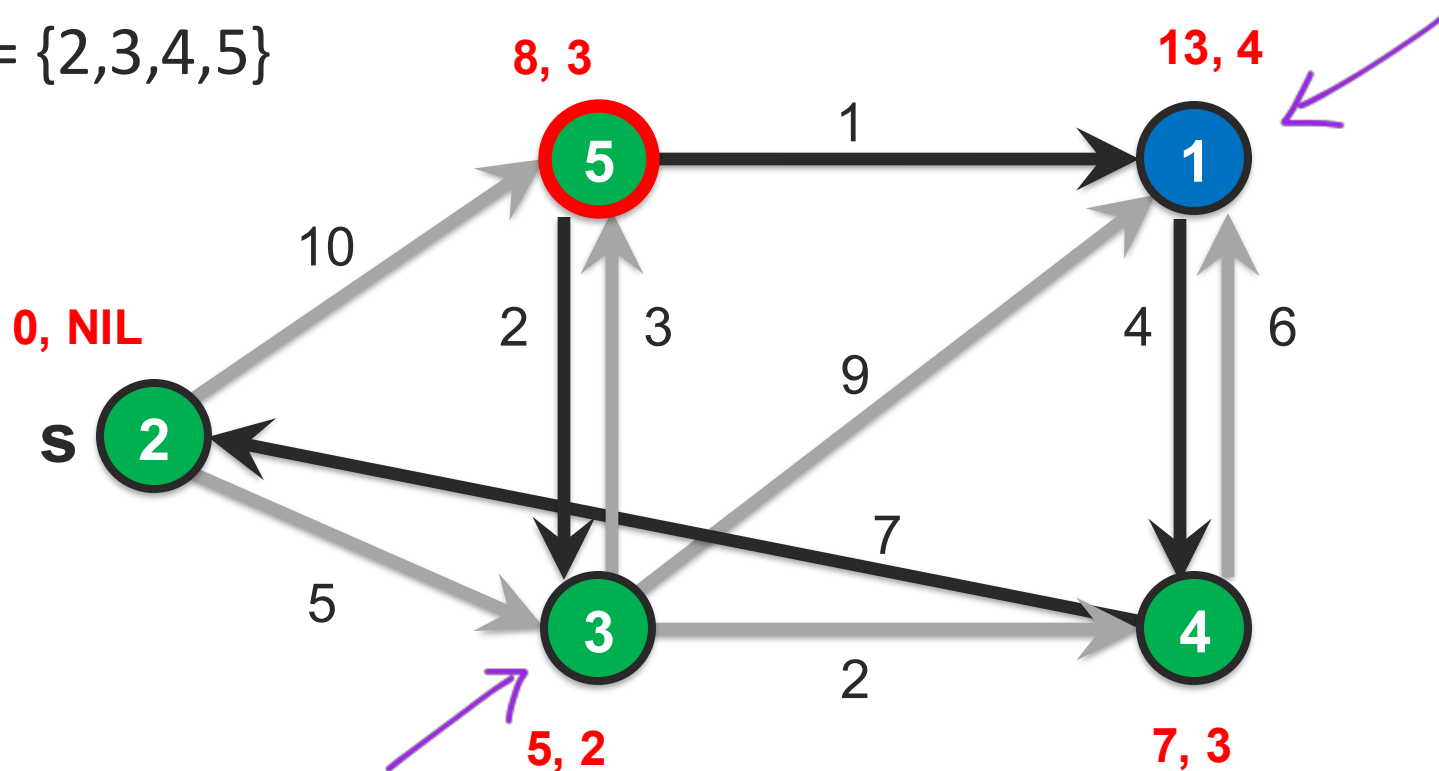
$S = \{2,3,4,5\}$



Estrazione da Q del vertice 5

[Esempio]

$S = \{2, 3, 4, 5\}$

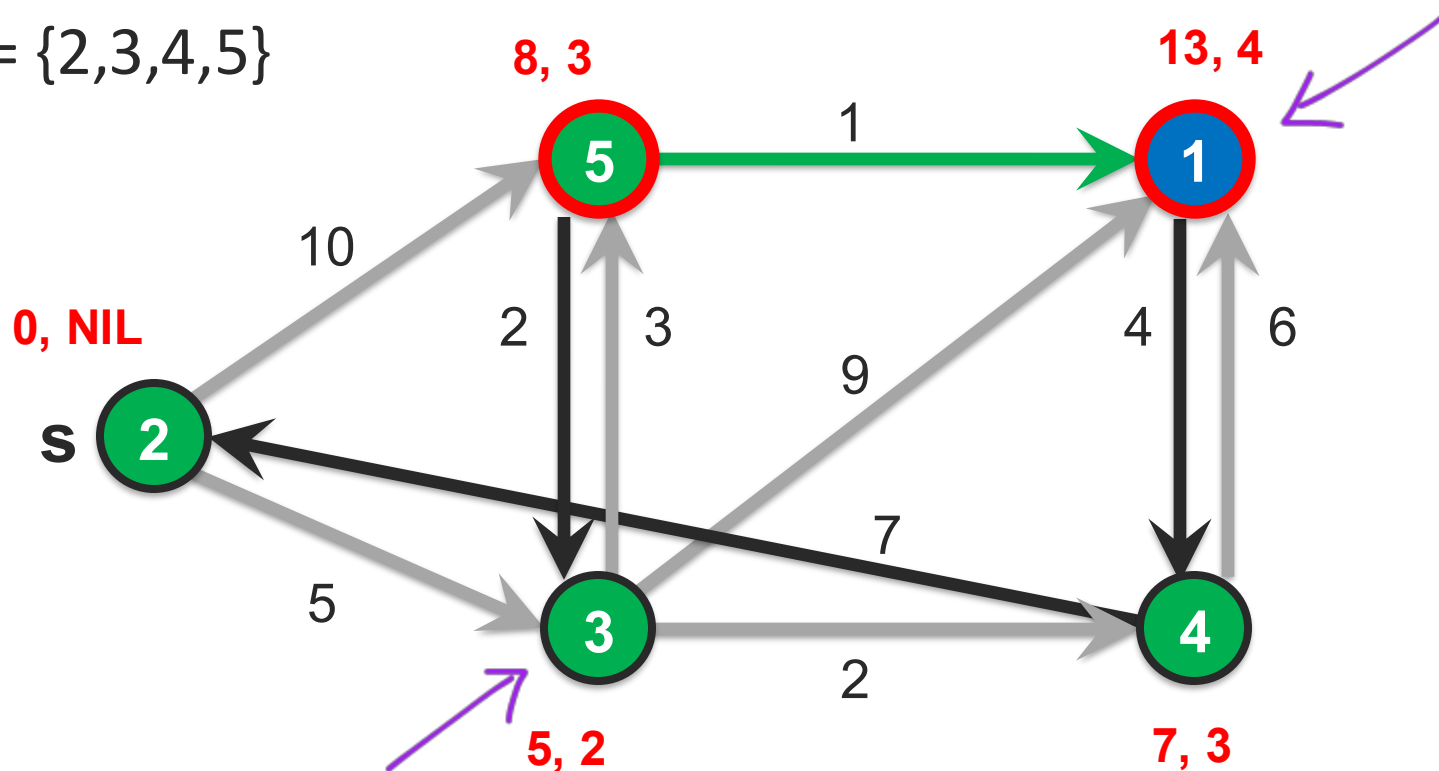


Estrazione da Q del vertice 5

Adiacenti $\rightarrow 1, 3$

[Esempio]

$S = \{2, 3, 4, 5\}$

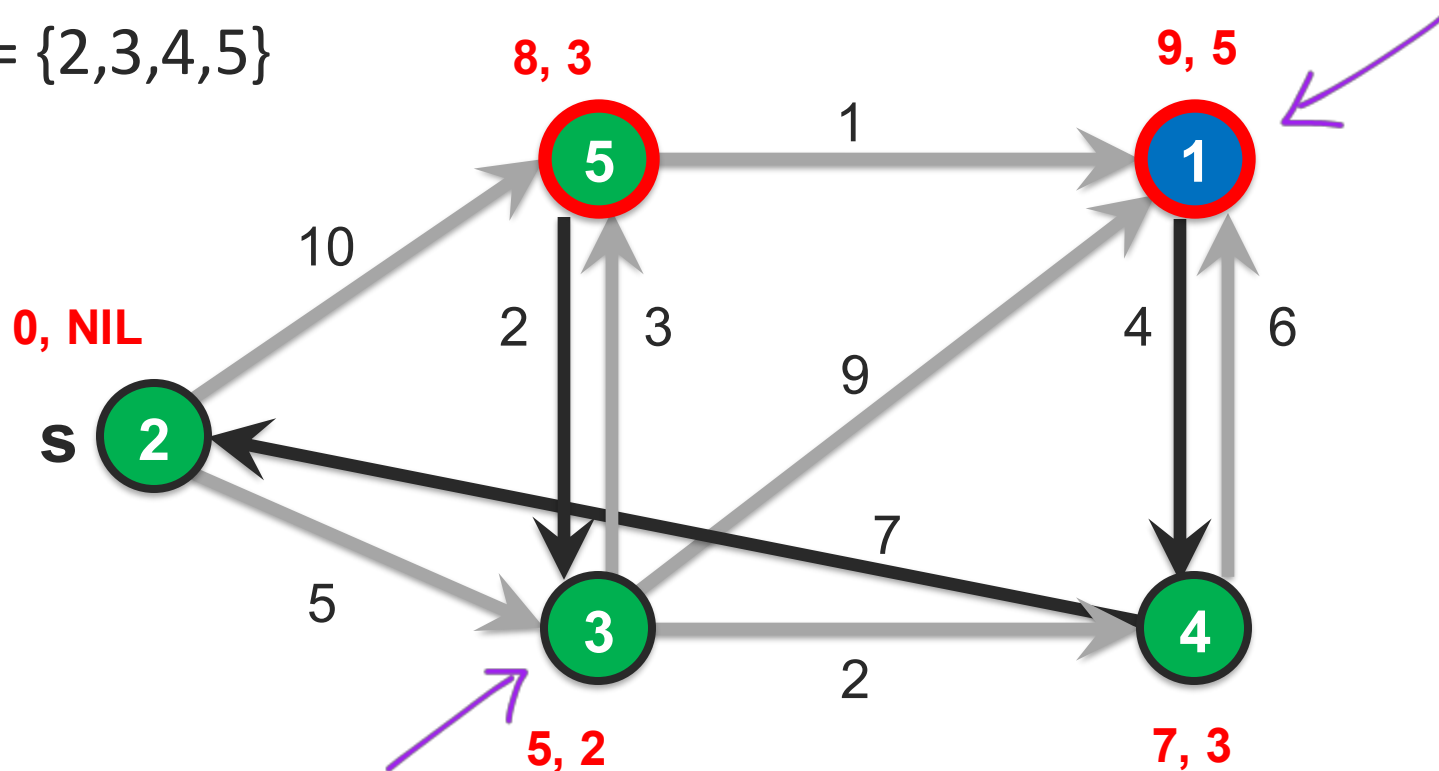


Estrazione da Q del vertice 5

Adiacenti → 1, 3

[Esempio]

$S = \{2, 3, 4, 5\}$

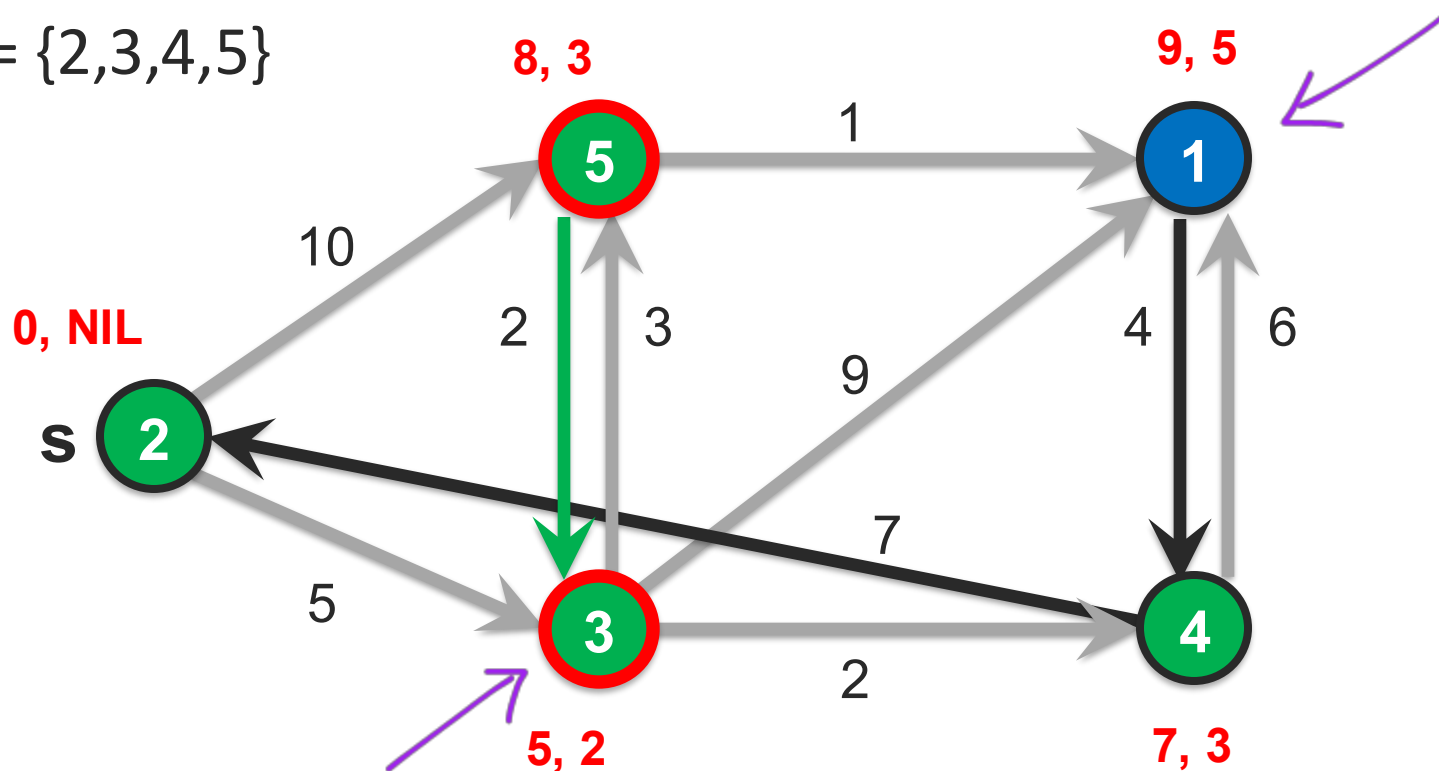


Estrazione da Q del vertice 5

Adiacenti $\rightarrow \underline{1}, 3$

[Esempio]

$S = \{2, 3, 4, 5\}$

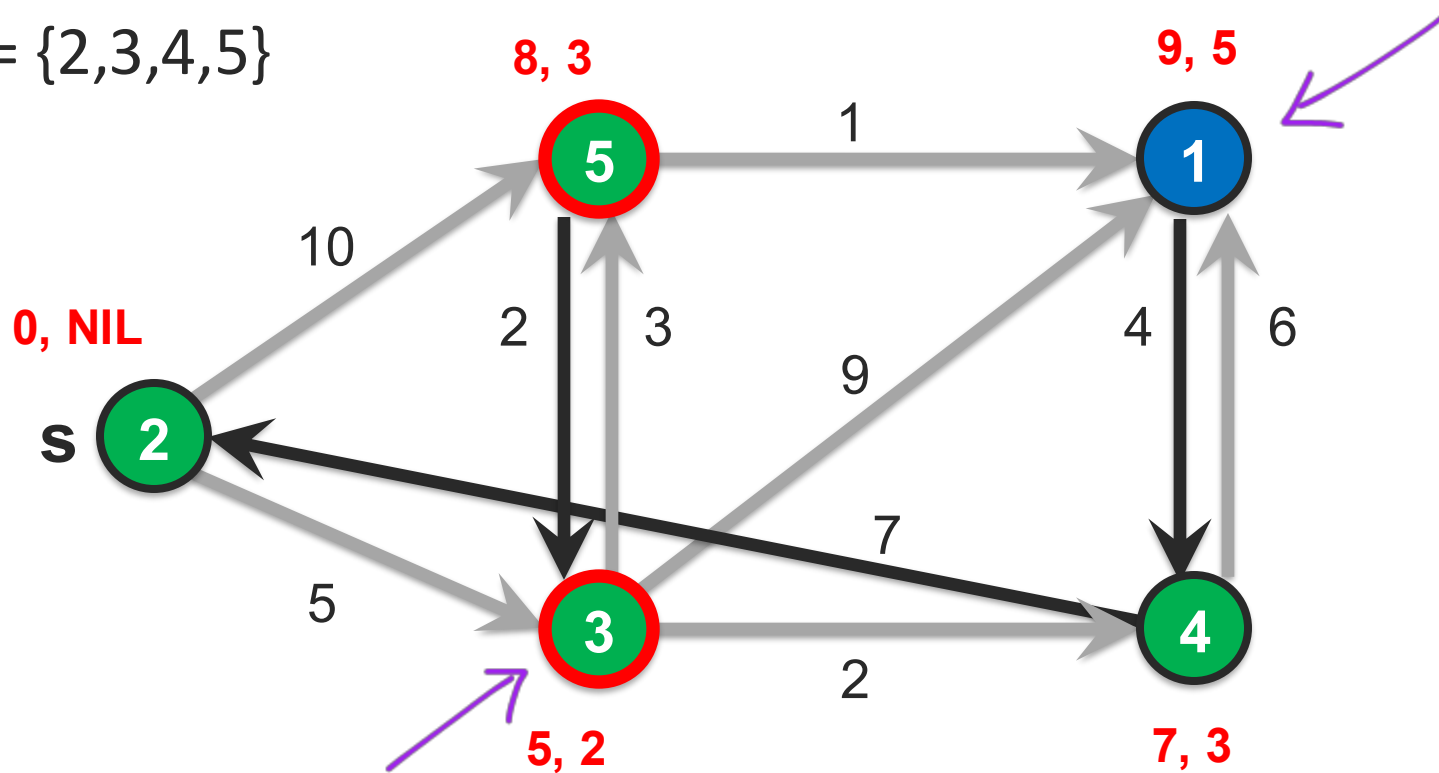


Estrazione da Q del vertice 5

Adiacenti $\rightarrow 1, \underline{3}$

[Esempio]

$S = \{2, 3, 4, 5\}$

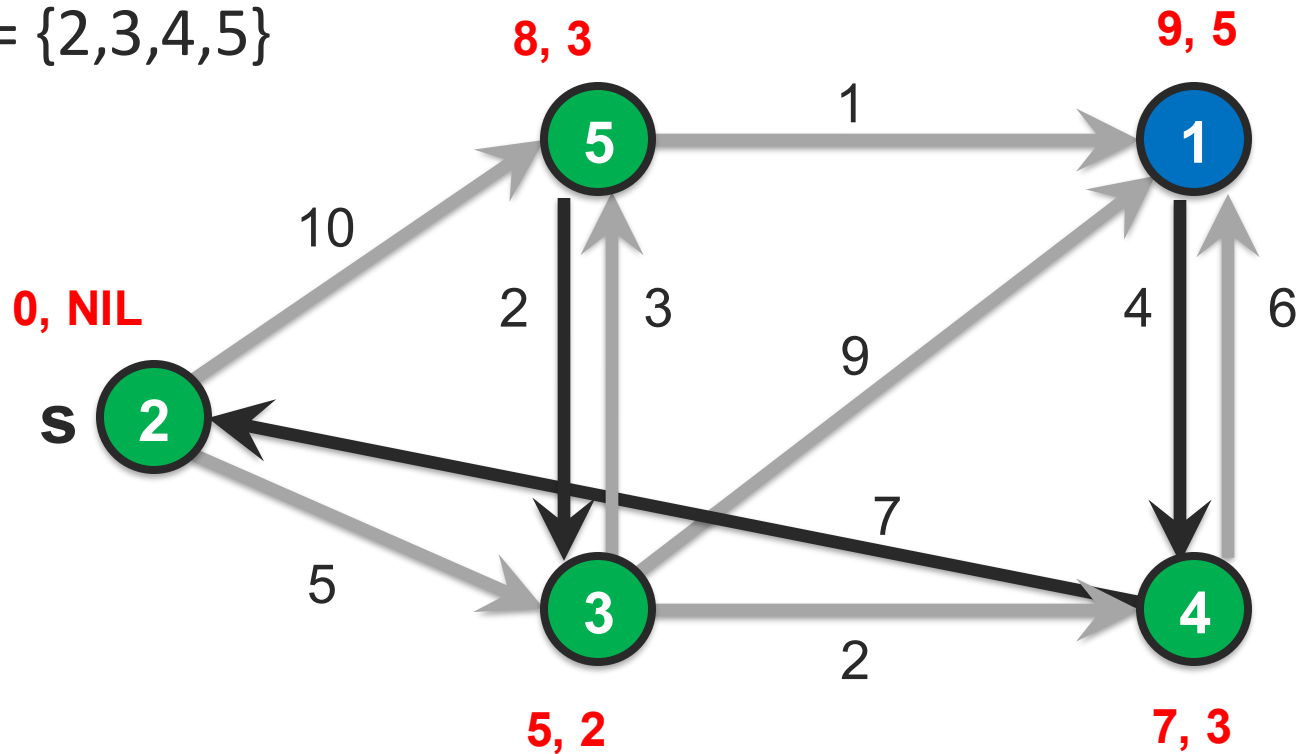


Estrazione da Q del vertice 5

Adiacenti $\rightarrow 1, \underline{3}$

[Esempio]

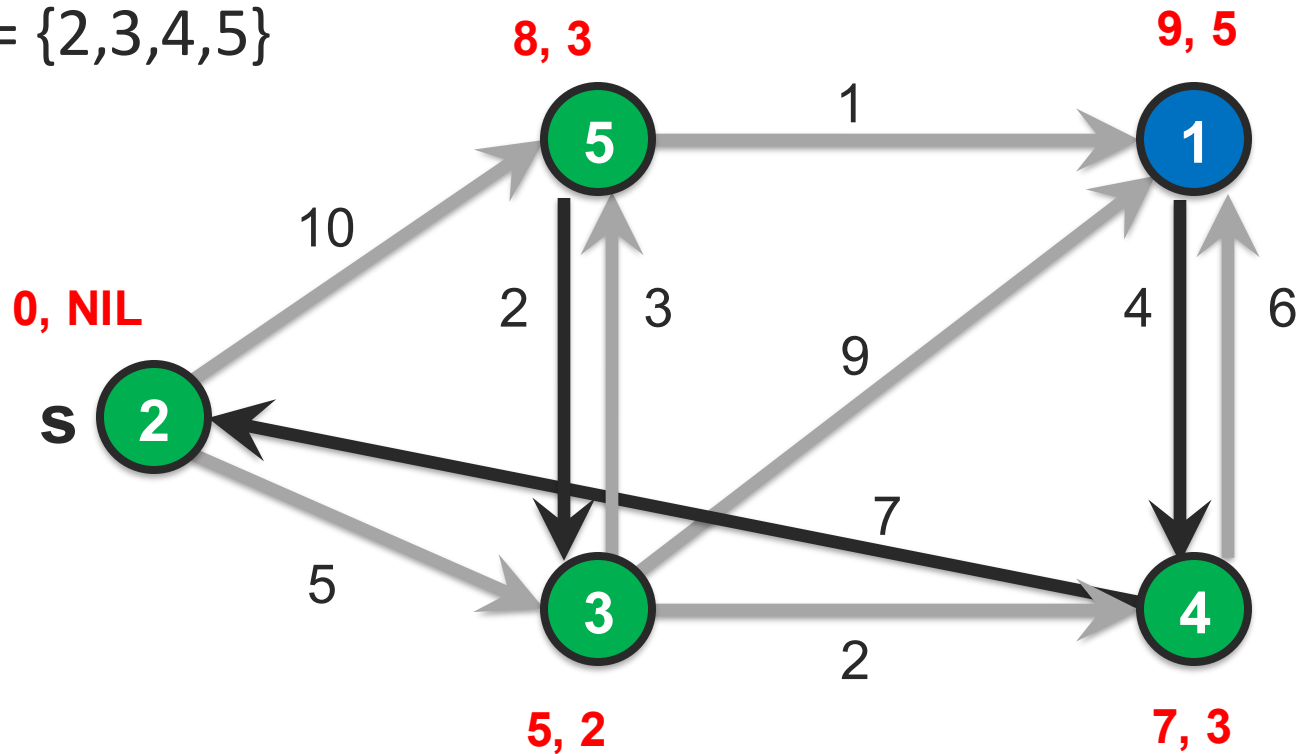
$S = \{2,3,4,5\}$



Estrazione da Q del vertice 5

[Esempio]

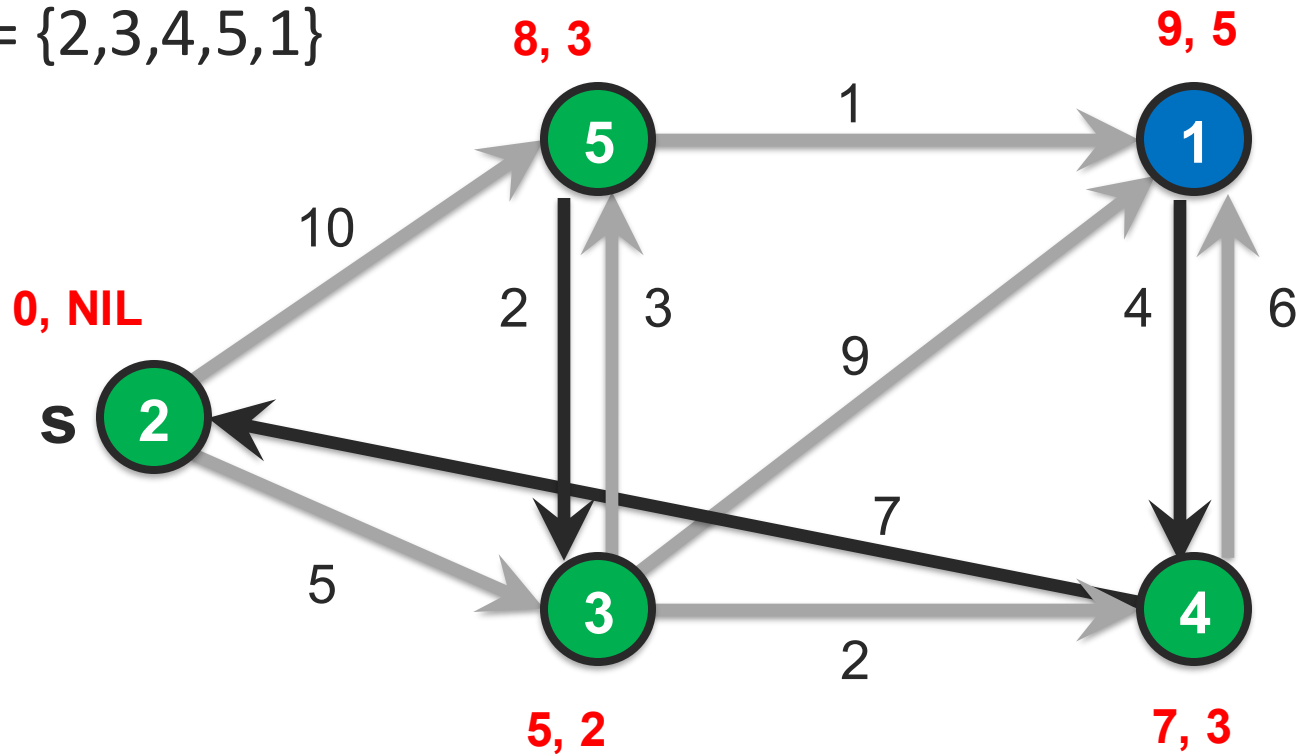
$S = \{2,3,4,5\}$



Estrazione da Q del vertice ?

[Esempio]

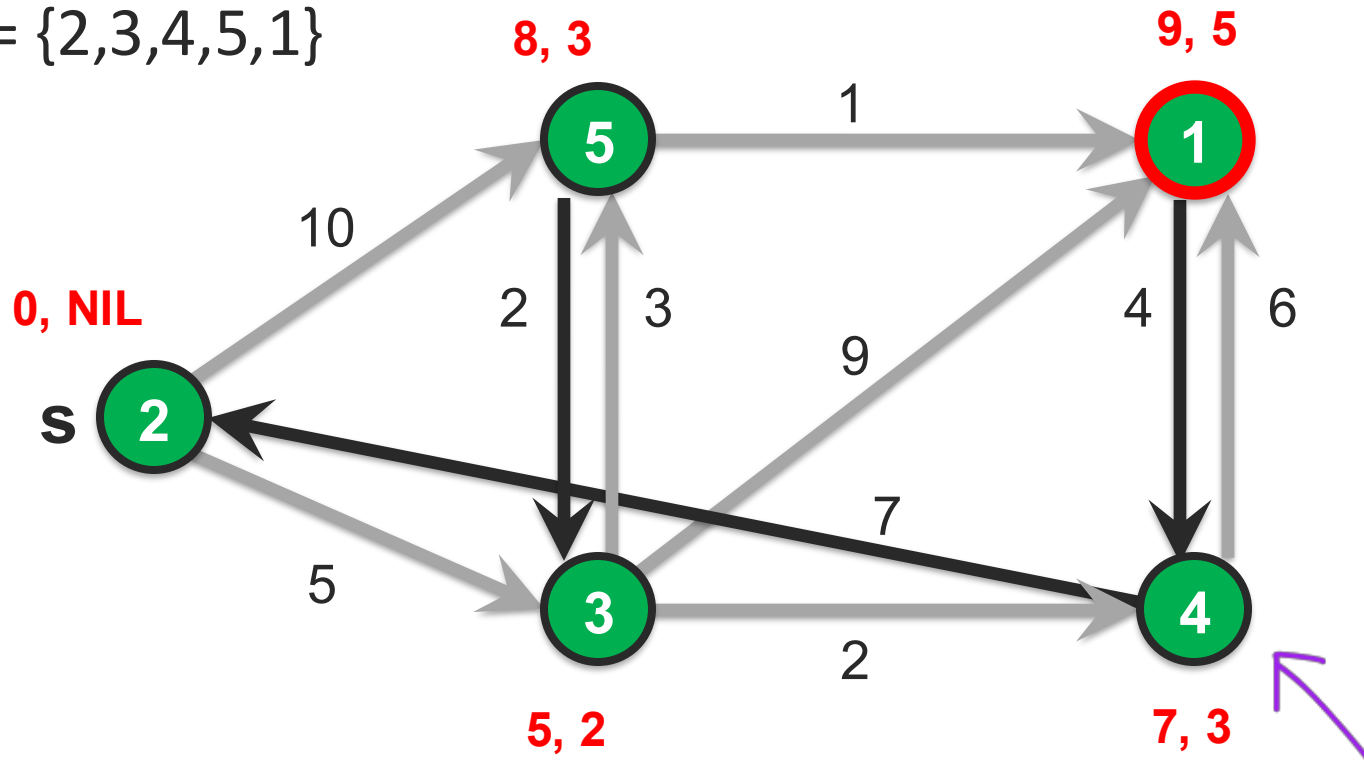
$S = \{2, 3, 4, 5, 1\}$



Estrazione da Q del vertice 1

[Esempio]

$S = \{2, 3, 4, 5, 1\}$

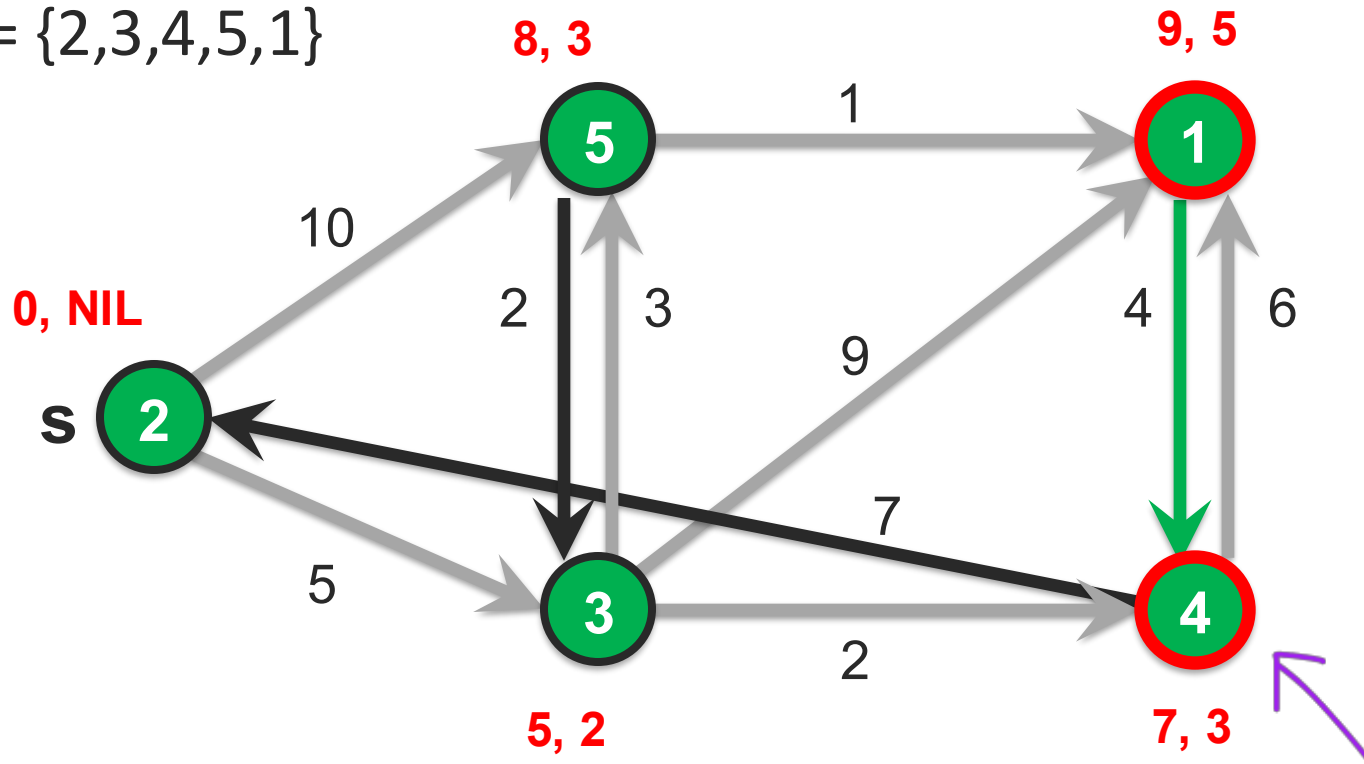


Estrazione da Q del vertice 1

Adiacenti → 4

[Esempio]

$S = \{2, 3, 4, 5, 1\}$

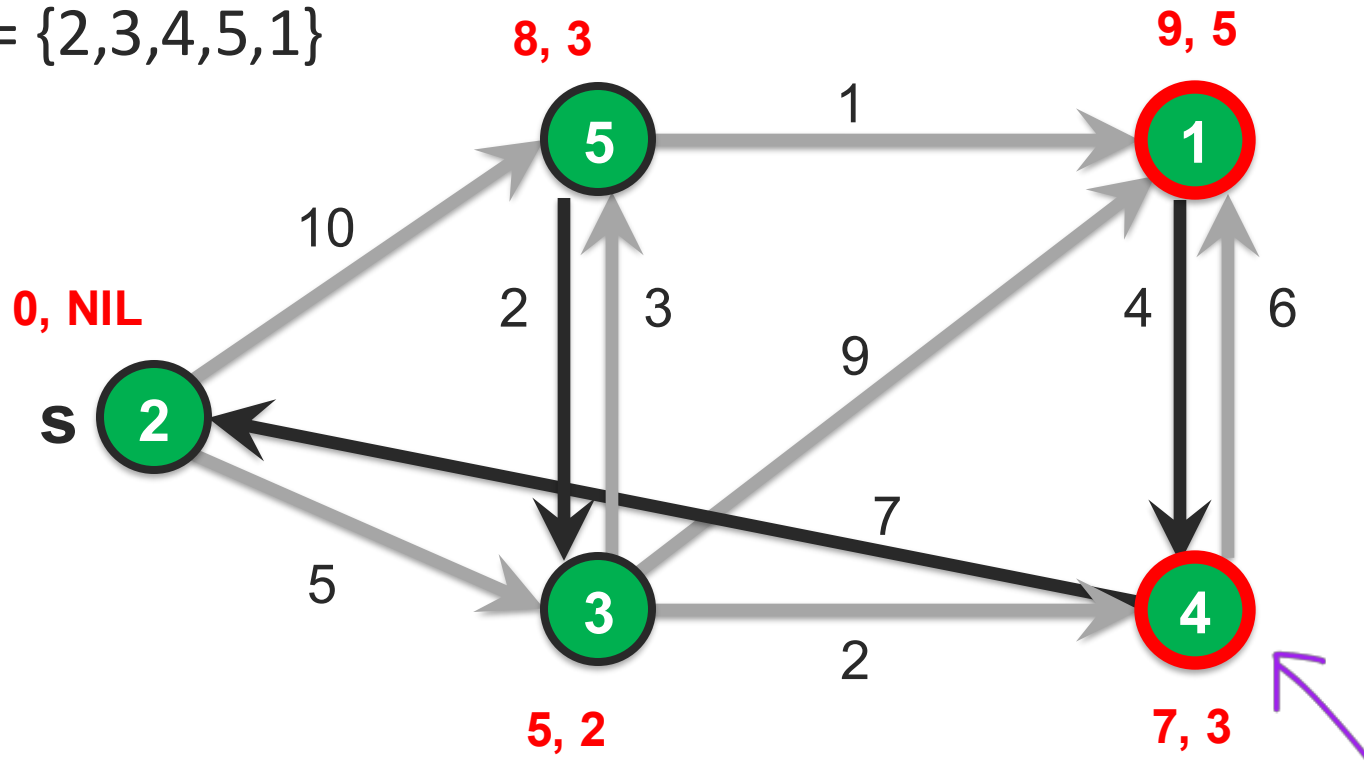


Estrazione da Q del vertice 1

Adiacenti → 4

[Esempio]

$S = \{2, 3, 4, 5, 1\}$

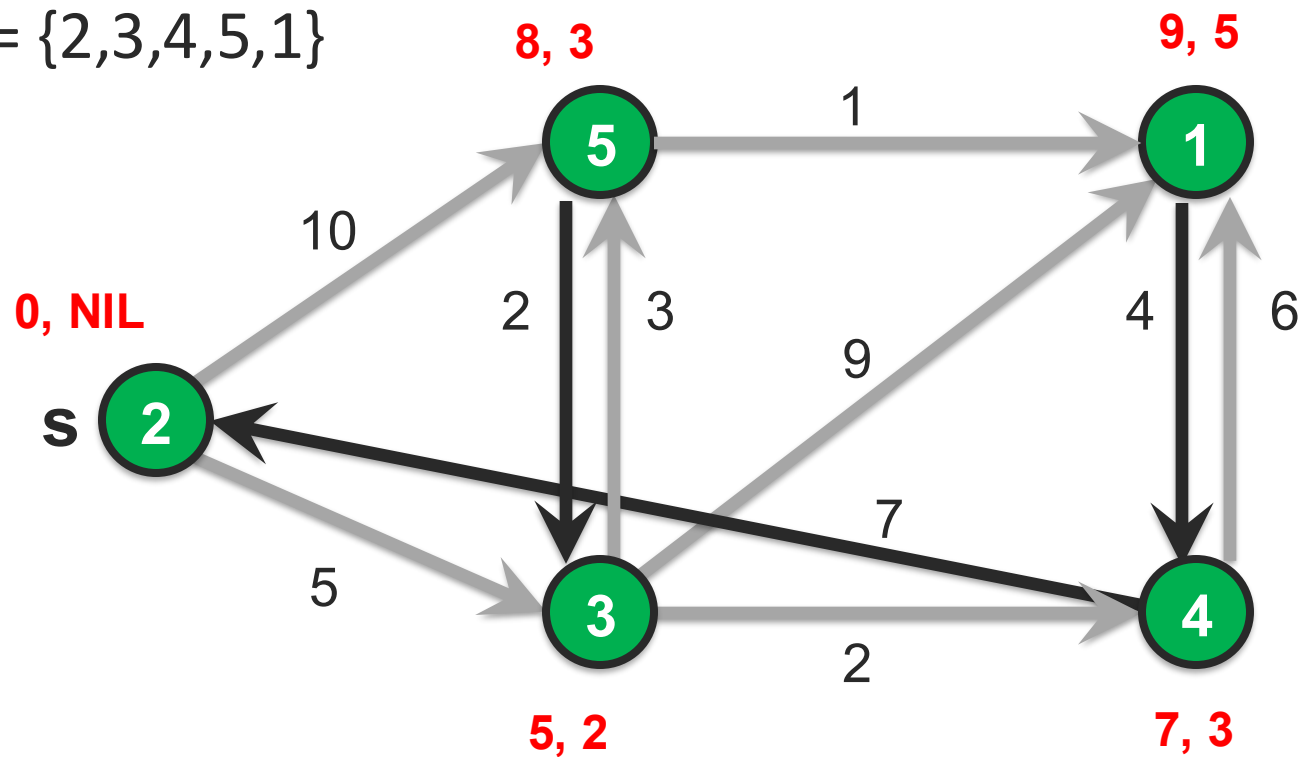


Estrazione da Q del vertice 1

Adiacenti → 4

[Esempio]

$S = \{2, 3, 4, 5, 1\}$



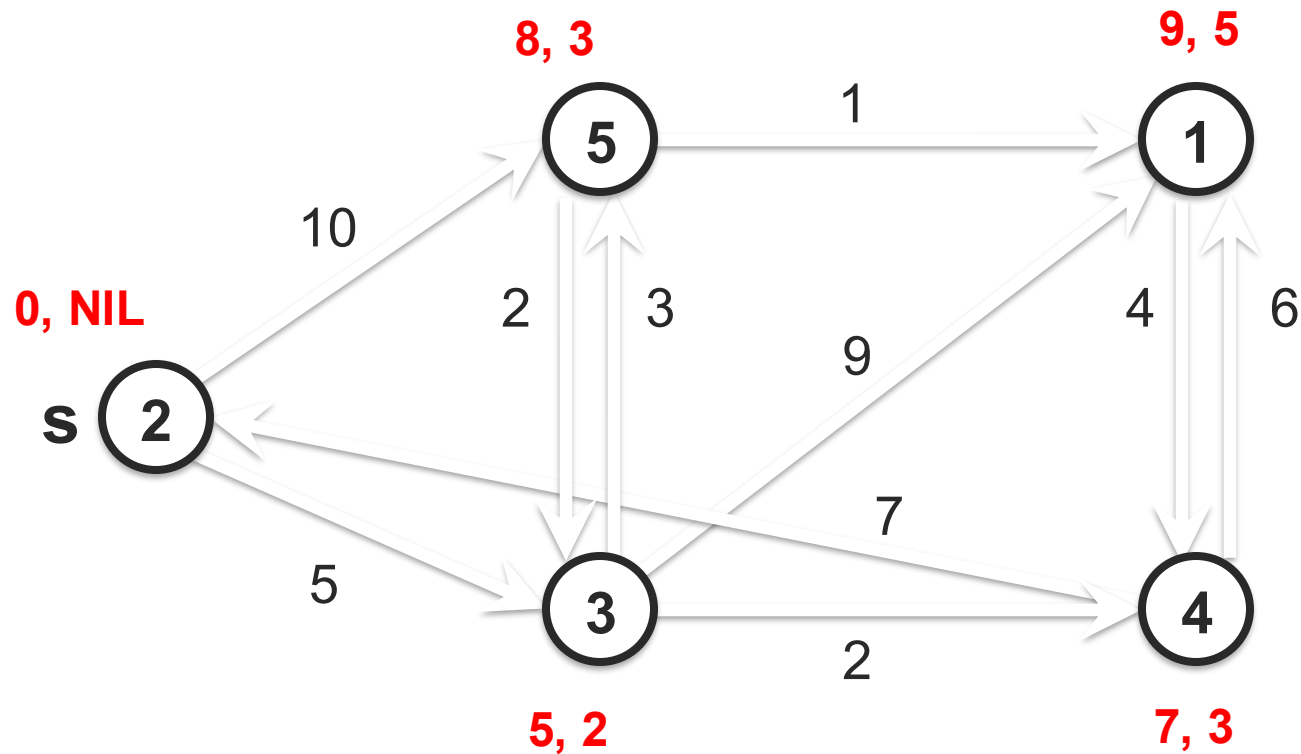
[Albero dei predecessori]

Albero dei predecessori (V_π, E_π)

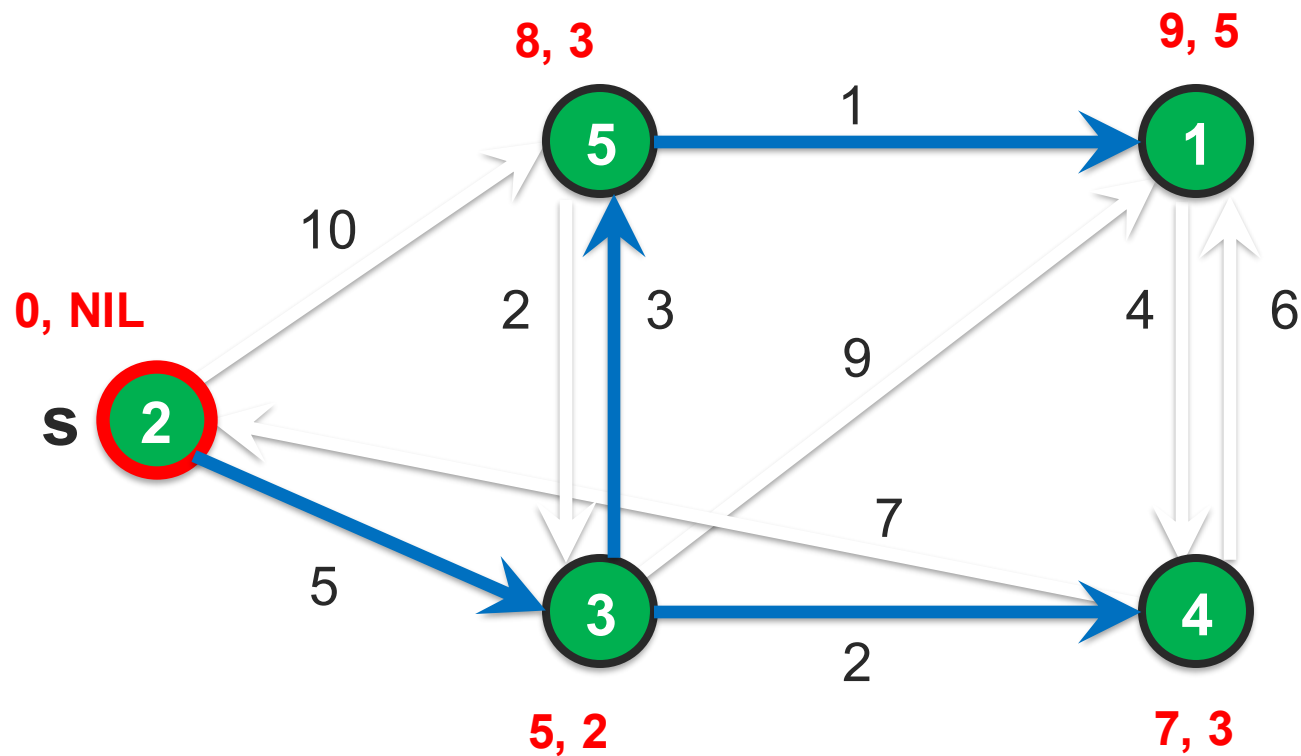
$V_\pi = \{v \text{ tale che } v \in V, v.d \neq +\infty\}$

$E_\pi = \{(v.\pi, v) \text{ tale che } v \in V_\pi, v \neq s\}$

[Albero dei predecessori]



[Albero dei predecessori]



$$V_{\pi} = \{1, 2, 3, 4, 5\}$$

$$E_{\pi} = \{ \longrightarrow \}$$

[Ricostruzione dei cammini]

$$P_{sv} = \langle s, \dots, v.\pi, v \rangle = \langle s, \dots, v.\pi \rangle + \langle v \rangle$$

Procedura ricorsiva

1. Ricostruisci il cammino minimo da **s** a **v.π**
2. Aggiungi **v**

```
Procedura stampa_cammino(v)
    if v.π ≠ NIL or v = s then
        if v.π ≠ NIL then
            stampa_cammino(v.π)
        print(v)
```

[Prova di correttezza]

Si deve dimostrare che al termine dell'algoritmo:

✓ $\forall x \in V, \mathbf{x.d} = \delta(s, x)$

~~✓ $\forall x \in V, \mathbf{x.\pi}$ è il predecessore di x nel cammino minimo $\mathbf{P}_{sx}$~~

TEOREMA. Si ha $\mathbf{u.d} = \delta(s, u)$ nel momento in cui u viene estratto da Q

CASO BASE

$\mathbf{u.d} = \delta(s, u)$, se $u = s$

PASSO INDUTTIVO

IPOTESI: $\mathbf{x.d} = \delta(s, x)$ per ogni $x \in S$

TESI: $\mathbf{u.d} = \delta(s, u)$ per il prossimo vertice u estratto da Q

[Prova di correttezza]

DIMOSTRAZIONE DEL PASSO INDUTTIVO

u prossimo vertice estratto da $Q \Rightarrow \mathbf{u.d} \leq \mathbf{u'.d}$ per ogni $u' \in Q$

(Per assurdo) u è il primo vertice estratto da Q tale che $\mathbf{u.d} \neq \delta(s,u)$

$\Rightarrow$ deve esistere qualche cammino da s a u , altrimenti $\mathbf{u.d} = \delta(s,u) = +\infty$

Sia:

- P_{su} il cammino minimo da s a u

[Prova di c

DIMOSTRAZIONE DEL PA

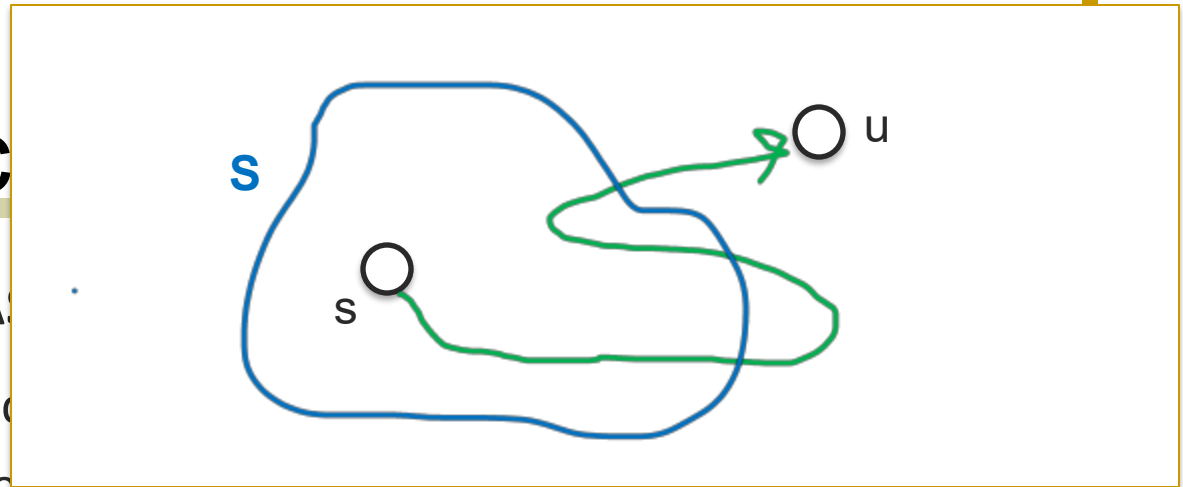
u prossimo vertice estratto c

(Per assurdo) u è il primo vertice estratto da Q tale che $u.d \neq \delta(s,u)$

$\Rightarrow$ deve esistere qualche cammino da s a u, altrimenti $u.d = \delta(s,u) = +\infty$

Sia:

- P_{su} il cammino minimo da s a u



[Prova di c

DIMOSTRAZIONE DEL PA

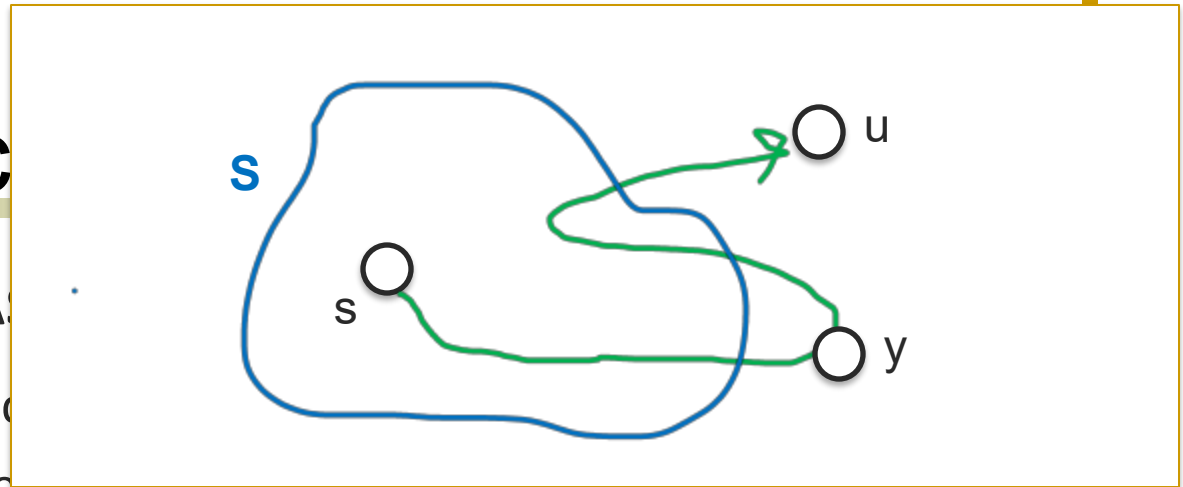
u prossimo vertice estratto c

(Per assurdo) u è il primo vertice estratto da Q tale che $u.d \neq \delta(s,u)$

$\Rightarrow$ deve esistere qualche cammino da s a u, altrimenti $u.d = \delta(s,u) = +\infty$

Sia:

- P_{su} il cammino minimo da s a u
- y il primo vertice di P_{su} non appartenente a S



[Prova di c

DIMOSTRAZIONE DEL PA

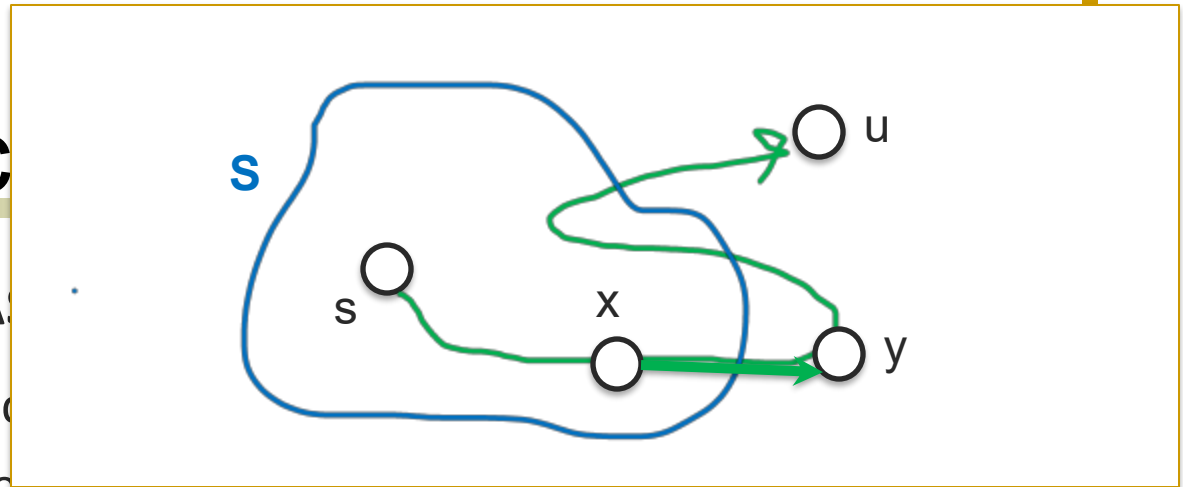
u prossimo vertice estratto c

(Per assurdo) u è il primo vertice estratto da Q tale che $u.d \neq \delta(s,u)$

$\Rightarrow$ deve esistere qualche cammino da s a u, altrimenti $u.d = \delta(s,u) = +\infty$

Sia:

- P_{su} il cammino minimo da s a u
- y il primo vertice di P_{su} non appartenente a S
- x predecessore di y in P_{su} (appartiene a S)



[Prova di c

DIMOSTRAZIONE DEL PA

u prossimo vertice estratto c

(Per assurdo) u è il primo vertice estratto da Q tale che $u.d \neq \delta(s,u)$

$\Rightarrow$ deve esistere qualche cammino da s a u, altrimenti $u.d = \delta(s,u) = +\infty$

Sia:

- P_{su} il cammino minimo da s a u
- y il primo vertice di P_{su} non appartenente a S
- x predecessore di y in P_{su} (appartiene a S)

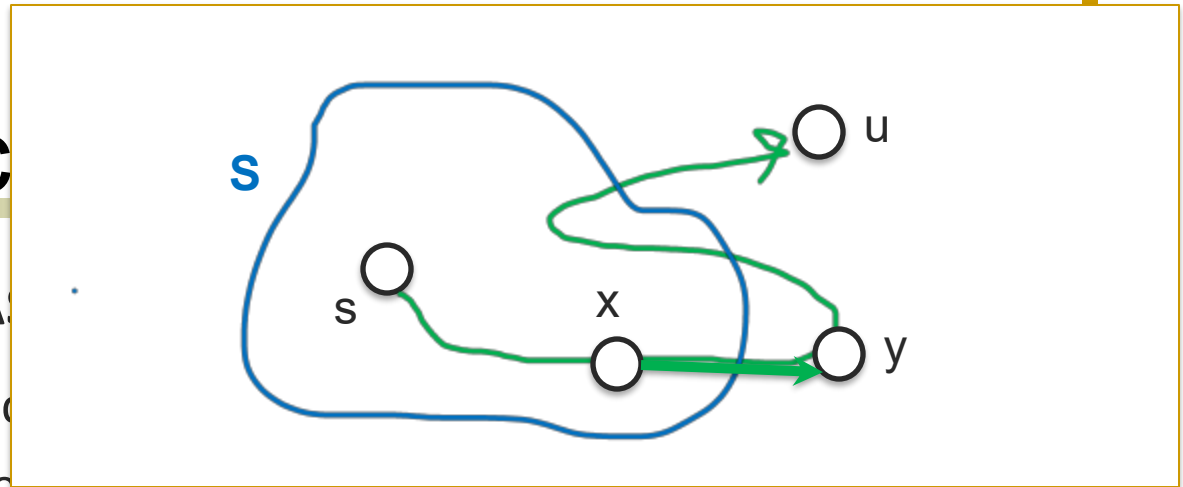
u è il primo vertice estratto tale che $u.d \neq \delta(s,u) \Rightarrow x.d = \delta(s,x)$ quando è stato estratto da Q e aggiunto a S

Il sottocammino di P_{su} da s a y coincide con il cammino minimo da s a y

(x,y) è stato rilassato nell'istante in cui x è stato aggiunto a S $\Rightarrow y.d = \delta(s,y)$

Cammino da s a y è sottocammino di $P_{su} \Rightarrow y.d = \delta(s,y) \leq \delta(s,u) \leq u.d$

$u.d \leq u'.d$ per ogni $u' \in Q \Rightarrow u.d \leq y.d \Rightarrow y.d = u.d \Rightarrow u.d = \delta(s,u)$



[Complessità in tempo]

Procedura DIJKSTRA (V, E, W, s)

$S \leftarrow$ empty set

$Q \leftarrow$ empty min_priority queue

INITIALIZE-SINGLE-SOURCE ($G = (V, E), s$)

foreach $v \in V$ **do**

 INSERT(Q, v)

while $Q \neq \emptyset$ **do**

$u \leftarrow$ EXTRACT-MIN(Q)

$S \leftarrow S \cup \{u\}$

foreach $v \in \text{adj}(u)$ **do**

 RELAX(u, v, W)

if $v.d$ è diminuito **then**

 DECREASE-KEY($Q, v, v.d$)

Q implementata con min-heap
binario e supponendo che G
sia connesso ($|E| > |V|$)

[Compl $O(|V|)$

Procedura DIJKSTRA (V, E, W, s)

$S \leftarrow$ empty set

$Q \leftarrow$ empty min_priority queue

INITIALIZE-SINGLE-SOURCE ($G = (V, E), s$)

foreach $v \in V$ **do**

 INSERT(Q, v)

while $Q \neq \emptyset$ **do**

$u \leftarrow$ EXTRACT-MIN(Q)

$S \leftarrow S \cup \{u\}$

foreach $v \in \text{adj}(u)$ **do**

 RELAX(u, v, W)

if $v.d$ è diminuito **then**

 DECREASE-KEY($Q, v, v.d$)

} $O(|V|)$

[Compl $O(|V|)$

Procedura DIJKSTRA (V, E, W, s)

$S \leftarrow$ empty set

$Q \leftarrow$ empty min_priority queue

INITIALIZE-SINGLE-SOURCE ($G = (V, E), s$)

foreach $v \in V$ **do**

 INSERT(Q, v)

while $Q \neq \emptyset$ **do**

$u \leftarrow$ EXTRACT-MIN(Q)

$S \leftarrow S \cup \{u\}$

foreach $v \in \text{adj}(u)$ **do**

 RELAX(u, v, W)

if $v.d$ è diminuito **then**

 DECREASE-KEY($Q, v, v.d$)

} $O(|V|)$

← $O(\log |V|)$

[Compl $O(|V|)$

Procedura DIJKSTRA (V, E, W, s)

$S \leftarrow$ empty set

$Q \leftarrow$ empty min_priority queue

INITIALIZE-SINGLE-SOURCE ($G = (V, E), s$)

foreach $v \in V$ **do**

 INSERT(Q, v)

} $O(|V|)$

while $Q \neq \emptyset$ **do**  $O(|V|)$ estrazioni da Q

$u \leftarrow$ EXTRACT-MIN(Q)  $O(\log |V|)$

$S \leftarrow S \cup \{u\}$

foreach $v \in \text{adj}(u)$ **do**

 RELAX(u, v, W)

if $v.d$ è diminuito **then**

 DECREASE-KEY($Q, v, v.d$)

[Compl $O(|V|) + O(|V| \log |V|)$

Procedura DIJKSTRA (V, E, W, s)

$S \leftarrow$ empty set

$Q \leftarrow$ empty min_priority queue

INITIALIZE-SINGLE-SOURCE ($G = (V, E), s$)

foreach $v \in V$ **do**

 INSERT(Q, v)

} $O(|V|)$

while $Q \neq \emptyset$ **do**  $O(|V|)$ estrazioni da Q

$u \leftarrow$ EXTRACT-MIN(Q)  $O(\log |V|)$

$S \leftarrow S \cup \{u\}$

foreach $v \in \text{adj}(u)$ **do**

 RELAX(u, v, W)

if $v.d$ è diminuito **then**

 DECREASE-KEY($Q, v, v.d$)

[Compl $O(|V|) + O(|V| \log |V|)$

Procedura DIJKSTRA (V, E, W, s)

$S \leftarrow$ empty set

$Q \leftarrow$ empty min_priority queue

INITIALIZE-SINGLE-SOURCE ($G = (V, E), s$)

foreach $v \in V$ **do**

 INSERT(Q, v)

} $O(|V|)$

while $Q \neq \emptyset$ **do**  $O(|V|)$ estrazioni da Q

$u \leftarrow$ EXTRACT-MIN(Q)  $O(\log |V|)$

$S \leftarrow S \cup \{u\}$

foreach $v \in \text{adj}(u)$ **do**

 RELAX(u, v, W)

if $v.d$ è diminuito **then**

 DECREASE-KEY($Q, v, v.d$)  $O(\log |V|)$

[Compl $O(|V|) + O(|V| \log |V|)$

Procedura DIJKSTRA (V, E, W, s)

$S \leftarrow$ empty set

$Q \leftarrow$ empty min_priority queue

INITIALIZE-SINGLE-SOURCE ($G = (V, E), s$)

foreach $v \in V$ **do**

 INSERT(Q, v)

} $O(|V|)$

while $Q \neq \emptyset$ **do**  $O(|V|)$ estrazioni da Q

$u \leftarrow$ EXTRACT-MIN(Q)  $O(\log |V|)$

$S \leftarrow S \cup \{u\}$

foreach $v \in \text{adj}(u)$ **do**  $O(|E|)$ iterazioni
 RELAX(u, v, W)
 if $v.d$ è diminuito **then**

 DECREASE-KEY($Q, v, v.d$)  $O(\log |V|)$

[Compl $O(|V|) + O(|V| \log |V|) + O(|E| \log |V|)$

Procedura DIJKSTRA (V, E, W, s)

$S \leftarrow$ empty set

$Q \leftarrow$ empty min_priority queue

INITIALIZE-SINGLE-SOURCE ($G = (V, E), s$)

foreach $v \in V$ **do**

 INSERT(Q, v)

} $O(|V|)$

while $Q \neq \emptyset$ **do**  $O(|V|)$ estrazioni da Q

$u \leftarrow$ EXTRACT-MIN(Q)  $O(\log |V|)$

$S \leftarrow S \cup \{u\}$

foreach $v \in \text{adj}(u)$ **do**  $O(|E|)$ iterazioni
 RELAX(u, v, W)
 if $v.d$ è diminuito **then**

 DECREASE-KEY($Q, v, v.d$)  $O(\log |V|)$

[Compl ~~$O(|V|)$~~ + $O(|V| \log |V|)$ + $O(|E| \log |V|)$]

Procedura DIJKSTRA (V, E, W, s)

$S \leftarrow$ empty set

$Q \leftarrow$ empty min_priority queue

INITIALIZE-SINGLE-SOURCE ($G = (V, E), s$)

foreach $v \in V$ **do**

 INSERT(Q, v)

} $O(|V|)$

while $Q \neq \emptyset$ **do**  $O(|V|)$ estrazioni da Q

$u \leftarrow$ EXTRACT-MIN(Q)  $O(\log |V|)$

$S \leftarrow S \cup \{u\}$

foreach $v \in \text{adj}(u)$ **do**  $O(|E|)$ iterazioni
 RELAX(u, v, W)
 if $v.d$ è diminuito **then**

 DECREASE-KEY($Q, v, v.d$)  $O(\log |V|)$

[Compl $O(|V| \log |V|) + O(|E| \log |V|)$

Procedura DIJKSTRA (V, E, W, s)

$S \leftarrow$ empty set

$Q \leftarrow$ empty min_priority queue

INITIALIZE-SINGLE-SOURCE ($G = (V, E), s$)

foreach $v \in V$ **do**

 INSERT(Q, v)

} $O(|V|)$

while $Q \neq \emptyset$ **do**  $O(|V|)$ estrazioni da Q

$u \leftarrow$ EXTRACT-MIN(Q)  $O(\log |V|)$

$S \leftarrow S \cup \{u\}$

foreach $v \in \text{adj}(u)$ **do**  $O(|E|)$ iterazioni
 RELAX(u, v, W)
 if $v.d$ è diminuito **then**

 DECREASE-KEY($Q, v, v.d$)  $O(\log |V|)$

[Compl $O(|E| \log |E|) + O(|E| \log |E|)$

Procedura DIJKSTRA (V, E, W, s)

$S \leftarrow$ empty set

$Q \leftarrow$ empty min_priority queue

INITIALIZE-SINGLE-SOURCE ($G = (V, E), s$)

foreach $v \in V$ **do**

 INSERT(Q, v)

} $O(|V|)$

while $Q \neq \emptyset$ **do**  $O(|V|)$ estrazioni da Q

$u \leftarrow$ EXTRACT-MIN(Q)  $O(\log |V|)$

$S \leftarrow S \cup \{u\}$

foreach $v \in \text{adj}(u)$ **do**  $O(|E|)$ iterazioni
 RELAX(u, v, W)
 if $v.d$ è diminuito **then**

 DECREASE-KEY($Q, v, v.d$)  $O(\log |V|)$

[Compl $O(|E| \log |E|)$

Procedura DIJKSTRA (V, E, W, s)

$S \leftarrow$ empty set

$Q \leftarrow$ empty min_priority queue

INITIALIZE-SINGLE-SOURCE ($G = (V, E), s$)

foreach $v \in V$ **do**

 INSERT(Q, v)

} $O(|V|)$

while $Q \neq \emptyset$ **do**  $O(|V|)$ estrazioni da Q

$u \leftarrow$ EXTRACT-MIN(Q)  $O(\log |V|)$

$S \leftarrow S \cup \{u\}$

foreach $v \in \text{adj}(u)$ **do**  $O(|E|)$ iterazioni
 RELAX(u, v, W)
 if $v.d$ è diminuito **then**

 DECREASE-KEY($Q, v, v.d$)  $O(\log |V|)$